

Renesas RA8 系列

IPC 双核 MCU 入门指南

引言

本应用笔记介绍 RA8 双核 MCU 上的核间通信（IPC），重点阐述 IPC 在实现两个 CPU 核之间无缝协调与数据交换方面的重要作用。它说明了 RA8 双核的 IPC 架构，描述了关键的通信机制，包括硬件信号量、消息 FIFO 以及通过可屏蔽和不可屏蔽中断（NMI）进行中断处理。

本指南提供使用 Renesas 灵活软件包（FSP）配置、实现和调试 IPC 的实操说明，并包含展示实际用例的参考工程。无论使用 FreeRTOS 等实时操作系统（RTOS）还是裸机环境开发，本文档都能为开发者提供在 RA8 双核平台上构建安全、可靠、高性能双核应用所需的基础知识和工具。

本应用笔记涵盖以下内容：

- 双核系统架构：概述 RA8 双核 MCU 结构、CPU 选择、配置和资源分配，以实现高效双核运行。
- IPC 机制与实现：说明不同的 IPC 通信方法，包括硬件信号量、消息 FIFO 和中断，并提供示例代码。
- 同步与数据交换：使用硬件信号量、NMI、消息 FIFO 以及用户级封装函数调用 IPC HAL 驱动来管理核间通信的技术。
- 示例工程与参考应用：展示 IPC 技术的实际实现，包括基于 FreeRTOS 的 IPC 以及硬件信号量/消息 FIFO 示例。

所需资源

- 灵活软件包（FSP）v6.0.0 或更高版本 注意：不同板卡的 FSP 支持情况如下所列。
- RA8P1（需要 FSP 6.0.0 或更高版本）。
- RA8T2（需要 FSP 6.1.0 或更高版本）。
- RA8D2 和 RA8M2（需要 FSP 6.2.0 或更高版本）。
- Renesas e² studio - 版本：2025-04.1 或更高版本。

支持的 MCU 系列

截至本文档发布时，支持的 MCU 系列包括：

- RA8P1 系列
- RA8T2 系列
- RA8D2 系列
- RA8M2 系列

注意：本应用笔记及随附的应用工程以 RA8P1 双核 MCU 为参考开发。但由于 RA8T2、RA8D2 和 RA8M2 MCU 共享相同的架构，这些资源同样可作为其他双核 MCU 的宝贵参考。这些应用工程可作为参考，用于为其他 RA8 双核 MCU 重新创建应用工程。

Contents

1. 引言	3
-------------	---

2. 双核系统架构	3
2.1 RA8 双核 MCU 架构概述	3
2.2 主 CPU 与辅助 CPU 工程选择	4
2.3 两个核的设置与配置	5
2.3.1 使用 FSP Configurator 进行双核 MCU 配置	5
2.3.2 灵活的核心分配	5
2.3.3 RA8 双核的启动行为	5
2.4 RA8 双核 MCU 资源分配	5
2.4.1 外设共享与同步	5
2.4.2 共享内存资源	5
2.4.3 通过 solution.xml 进行内存配置	5
3. IPC 模块	6
3.1 RA8 双核 MCU IPC 概述	6
3.1.1 硬件信号量 (Semaphore)	6
3.1.2 核间中断 (IPI)	7
3.1.3 消息 FIFO	7
3.1.4 RA8 双核 MCU IPC 寄存器架构	8
3.2 使用 FSP 实现 IPC 模块	10
3.2.1 IPC API 概述: 关键组件与数据结构	10
4. 同步与数据交换	11
4.1 双核通信的硬件信号量机制	11
4.2 IPC 的可屏蔽与不可屏蔽中断	11
4.3 用于核间数据交换的消息 FIFO	11
5. IPC 的 FSP 支持	11
5.1 使用 FSP Configurator 设置 IPC	12
5.2 使用说明: IPC 中断与通道配置	14
5.2.1 中断与回调行为	14
5.2.2 核间通道匹配	14
5.2.3 IPC 硬件信号量支持	15
5.2.4 IPC 的 NMI 支持	15
6. 示例工程与参考应用	16
6.1 IPC 实现示例代码	16
6.2 RA8P1 MCU IPC 示例代码实现	18
6.3 在双核环境中使用 FreeRTOS 进行 IPC	19
7. 运行应用工程	19
7.1 导入工程	19
7.2 构建工程	20
7.3 下载、运行与验证工程	21

7.4 验证应用	23
8. 调试与故障排除.....	27
9. 后续步骤	27
10. 参考文献	27
修订历史	30

1. 引言

Renesas RA 双核 MCU 中的 IPC 机制通过支持硬件共享、内存访问同步和数据交换，实现两个 CPU 核之间的高效协调。它提供多种通信方法，包括硬件信号量、FIFO 缓冲区和中断事件，确保 CPU0 与 CPU1 核之间顺畅可靠的交互。

为防止资源冲突并维护系统完整性，IPC 机制使用多个锁和硬件信号量控制来强制对共享外设和内存的互斥访问。根据应用需求，开发者可选择专用或共享的硬件信号量。

在数据交换方面，IPC 包含基于 FIFO 的通信通道，便于高效、结构化的数据流动：

- CPU0 的只写 FIFO 和 CPU1 的只读 FIFO，使 CPU0 能够向 CPU1 发送数据。
- CPU1 的只写 FIFO 和 CPU0 的只读 FIFO，使 CPU1 能够向 CPU0 发送数据。

从而支持两个核之间的双向通信。

这些特性为双核应用提供了一个健壮的框架，可实现 CPU0 与 CPU1 之间可靠的资源划分、同步操作和高效数据共享。

2. 双核系统架构

RA 双核 MCU 在单芯片内集成两个处理核，以提升性能、并行处理能力和功耗效率。这些 MCU 常用于实时应用、物联网设备和工业自动化系统。

Renesas RA8 双核 MCU 采用异构架构，Cortex-M85（CM85）核配置为核 0，Cortex-M33（CM33）核配置为核 1。核 0 用于计算密集型操作，而核 1 负责系统控制和外设交互。

2.1 RA8 双核 MCU 架构概述

Renesas RA 双核 MCU 的系统架构旨在实现核与所连接外设之间的高效通信和数据交换。系统主要采用共享总线架构，两个核通过连接到内存和外设的标准系统总线进行接口。为维持有序访问并防止冲突，实现了仲裁机制来管理资源共享并公平地确定请求优先级。此外，专用总线用于访问指令紧耦合内存（ITCM）和数据紧耦合内存（DTCM），实现低延迟、高速的内存操作。

为管理共享资源访问并协调两个核之间的操作，RA8 双核架构利用 IPC 机制，包括 FIFO 缓冲区、中断信号、共享内存区域和硬件信号量，这些机制共同支持同步和高效数据交换。IPC 确保两个 CPU 能够有效协同工作，同时保持系统稳定性和性能。

底层双核总线架构建立在标准互连总线之上，如 AXI、AHB 和 APB 总线，用于实现 Cortex-M85 与 Cortex-M33 核之间以及核与内存和外设之间的通信。

RA8 双核 MCU 配备一颗运行频率高达 1 GHz 的高性能 Arm® Cortex®-M85 核，以及一颗运行频率高达 250 MHz 的 Arm® Cortex®-M33 核，并提供以下关键特性：

- 最高 1 MB MRAM，用于非易失性存储器存储
- 2 MB SRAM，包括：
 - Cortex-M85 核配备 256 KB TCM RAM
 - Cortex-M33 核配备 128 KB TCM RAM
- 1.664 MB 用户可访问 SRAM
- 集成 Arm® Ethos™-U55 神经网络处理单元（NPU）
- 八线串行外设接口（OSPI），用于高速闪存访问
- 三层以太网交换模块（ESWM）、USB 全速（USBFS）、USB 高速（USBHS）和 SD/MMC 主机接口
- 图形 LCD 控制器（GLCDC），用于高级显示支持
- 2D 绘图引擎（DRW），用于硬件加速图形渲染
- 支持 MIPI DSI/CSI 接口，用于显示器和摄像头连接
- 丰富的模拟外设集合
- 全面的安全与功能安全特性，确保可靠运行

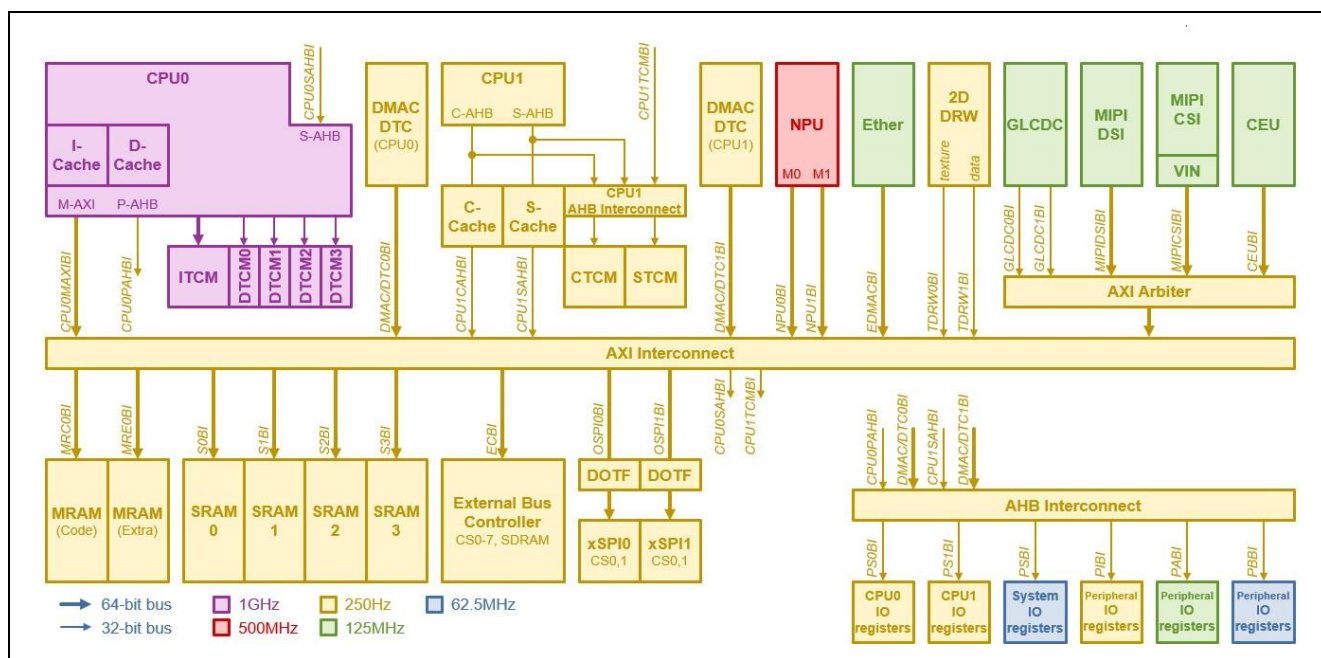


图 1. RA8 双核系统架构概述

有关 RA8 双核 MCU 架构的更多信息，请参阅《使用 RA8 双核 MCU 开发》(R01AN7881) 应用笔记。

2.2 主 CPU 与辅助 CPU 工程选择

e² studio 结合 FSP 和 BSP，通过单次运行工程创建向导即可简化双核工程的创建流程。此操作通过"Renesas RA FSP Solution"工程创建选项完成。

当目标为双核器件时，向导会自动生成三个工程：

1. **CPU0** 工程（主核）：包含主核的应用代码。
2. **CPU1** 工程（从核）：包含从核的应用代码。
3. **Solution**（容器）工程：作为管理两个 **CPU** 工程的顶层容器，负责共享配置、工程组织和整体方案结构。

FSP 和 BSP 共同为 CPU0 和 CPU1 提供默认配置，使开发者能够专注于应用开发，同时确保两个核充分集成以部署到目标 MCU。

2.3 两个核的设置与配置

2.3.1 使用 FSP Configurator 进行双核 MCU 配置

Renesas RA 系列双核 MCU 的设置与配置通过 FSP Configurator 在 e² studio 中进行管理。

当创建双核 RA FSP Solution 工程时，该工具默认自动生成两个独立的应用工程，分别用于 CPU0（Cortex-M85）和 CPU1（Cortex-M33）。

2.3.2 灵活的核心分配

虽然默认配置将 Cortex-M85（CPU0）分配为主 CPU、Cortex-M33（CPU1）分配为辅助 CPU，但开发者可根据应用需求重新分配核心角色。

2.3.3 RA8 双核的启动行为

在 RA8 双核 MCU 上，主 CPU 可通过启动固件命令配置，并在系统复位后首先开始执行。默认情况下，CPU0 配置为主核。

系统复位释放后：

辅助 CPU 保持在电源门控状态。

- 主 CPU 必须通过写 CPU_nACTCSR 寄存器显式激活辅助 CPU。
- 根据 CPU_nWAITCR.CPUWAIT 位的设置，辅助 CPU 将：
 - 进入等待状态（CPUWAIT），允许安全加载代码/数据到其 TCM 中，或
 - 立即开始程序执行。

注意：如果 CPU0 被配置为辅助核，在传输数据或释放等待状态之前必须验证其激活状态。主 CPU 不受 CPUWAIT 设置的影响。

2.4 RA8 双核 MCU 资源分配

在 RA8 双核 MCU 中，许多外设和内存资源在 CPU0（Cortex-M85）和 CPU1（Cortex-M33）两个核之间共享。这种共享架构可实现高效的硬件利用，但也需要谨慎的资源管理，特别是当两个 CPU 访问相同的外设或内存区域时。

2.4.1 外设共享与同步

MCU 的大多数外设模块（如定时器、通信接口（UART、SPI、I2C）和系统控制器）对两个核都是公共的。虽然这实现了灵活的任务分配，但开发者必须实现适当的同步机制以避免竞态条件和冲突。这通常通过使用硬件支持的 IPC 方法（如硬件信号量或锁）来实现，以协调访问并确保一次只有一个 CPU 与共享外设交互。

2.4.2 共享内存资源

内存资源（包括 MRAM、SRAM 和外部存储器接口（如 OSPI 或 SDRAM））也可由两个核访问。为支持根据应用需求自定义内存分段，RA8 双核 MCU 允许灵活分配内存区域。

2.4.3 通过 solution.xml 进行内存配置

在使用 Renesas FSP 进行工程设置期间，solution 工程中的 solution.xml 文件提供了基于 GUI 的内存配置界面。在 FSP Configurator 中打开此界面后，开发者可以：

- 定义选定内存块的起始和结束地址
- 根据角色将内存区域分配给任一 CPU
- 根据系统约束和性能需求微调内存大小和布局

内存配置如下图所示。这种灵活性使开发者能够设计均衡、高效且针对特定用例定制的应用，例如为 CM33 上的实时任务隔离内存，同时为 CM85 上的计算密集型操作保留高速内存。

Name	Start	Size	Core	Security
RAM_NS	0x32000000	0x200000		Non-secure
RAM	0x22000000	0x200000		Secure
RAM_CPU0_S	0x22000000	0x100000	CPU0	Secure
RAM_CPU0_C	0x22100000	0x0	CPU0	Non-secure Callable
RAM_CPU1_S	0x22100000	0x100000	CPU1	Secure
RAM_CPU1_C	0x22200000	0x0	CPU1	Non-secure Callable
FLASH_NS	0x12000000	0x100000		Non-secure
FLASH	0x02000000	0x100000		Secure
DATA_FLASH_NS	0x37000000	0x0		Non-secure
DATA_FLASH	0x27000000	0x0		Secure
SDRAM	0x68000000	0x8000000		
OSPI0_CS0	0x80000000	0x10000000		
OSPI0_CS1	0x90000000	0x10000000		
OSPI1_CS0	0x70000000	0x8000000		
OSPI1_CS1	0x78000000	0x8000000		
OPTION_SETTING_OFS0	0x02C9F040	0x4		Secure
OPTION_SETTING_OFS2	0x02C9F044	0x4		Secure
OPTION_SETTING_OFS2_CPU0_S	0x02C9F044	0x2	CPU0	Secure
OPTION_SETTING_OFS2_CPU1_S	0x02C9F046	0x2	CPU1	Secure
OPTION_SETTING_OFS1	0x12C9F4C0	0x4		Non-secure
OPTION_SETTING_OFS1_SEC	0x02C9F0C0	0x4		Secure
OPTION_SETTING_OFS1_SEL	0x02C9F120	0x4		Secure
OPTION_SETTING_OFS3	0x12C9F4C4	0x4		Non-secure
OPTION_SETTING_OFS3_SEC	0x02C9F0C4	0x4		Secure
OPTION_SETTING_OFS3_SEL	0x02C9F124	0x4		Secure
OPTION_SETTING_BPS	0x12C9F600	0x80		Non-secure
OPTION_SETTING_BPS_SEC	0x02C9F200	0x80		Secure
OPTION_SETTING_OTP_PBP5_SEC	0x02E07700	0x80		Secure
OPTION_SETTING_OTP_PBP5	0x12E07780	0x80		Non-secure
ITCM_NS	0x10000000	0x20000	CPU0	Non-secure
ITCM	0x00000000	0x20000	CPU0	Secure
DTCM_NS	0x30000000	0x20000	CPU0	Non-secure
DTCM	0x20000000	0x20000	CPU0	Secure
CTCM_NS	0x10000000	0x10000	CPU1	Non-secure
CTCM	0x00000000	0x10000	CPU1	Secure
STCM_NS	0x30000000	0x10000	CPU1	Non-secure
STCM	0x20000000	0x10000	CPU1	Secure

图 2. 使用 solution.xml 的 RA8 双核内存配置

3. IPC 模块

RA MCU 中的 IPC 机制实现了 RA8 双核 MCU 中两个处理器核之间的无缝资源共享与通信。它在协调任务、交换数据以及确保对共享硬件组件的安全访问方面发挥着至关重要的作用。

IPC 支持中断驱动的通信，允许一个处理器在另一个处理器中触发事件，这对于实时同步和信号传递尤为有用。

为确保对共享外设和内存的互斥访问，IPC 实现了一系列基于硬件的信号量机制，包括多个防止同时访问关键资源的锁寄存器，有助于维护数据完整性和系统稳定性。开发者可根据核间所需的隔离或交互程度，选择专用（分离）或共享的硬件信号量配置。

除了同步机制外，IPC 还利用 FIFO 缓冲区和共享内存实现简单高效的数据交换。这些通常配置为单向或双向方式。

3.1 RA8 双核 MCU IPC 概述

RA8 双核 MCU 在 CPU 内部集成了健壮的 IPC 系统，可实现两个处理器核 Cortex-M85（CPU0）和 Cortex-M33（CPU1）之间的高效协调、数据共享和硬件资源管理。

3.1.1 硬件信号量（Semaphore）

- IPC 提供 16 个硬件信号量锁来强制互斥，一次只允许一个 CPU 访问共享资源。
- 这些硬件信号量按安全和特权属性分类，每 8 个为一组，可根据应用需求实现受控且安全的访问。
- 这确保了数据一致性，避免了对共享外设或内存的争用。

3.1.2 核间中断 (IPI)

IPC 允许一个 CPU 在另一个 CPU 上触发中断事件，支持实时信号传递和事件处理。这些中断分为：

- 核间不可屏蔽中断 (NMI):
 - IPC0: 1 个因子 (CPU1 → CPU0) ◦ IPC1: 1 个因子 (CPU0 → CPU1)
 - 每个均包含安全和特权配置。
- 核间可屏蔽中断:
 - IPC00、IPC01、IPC10、IPC11: 每个支持 8 个因子，允许可屏蔽中断。
 - 每个因子可单独配置安全和特权属性。

这些中断类型提供了灵活的通知机制，适用于任务同步、核间信号传递和异常处理。

3.1.3 消息 FIFO

为支持核间快速、结构化的数据传输，IPC 提供消息 FIFO 通道，每个通道配置如下：

- 每个通道 4 级 FIFO
- 每级 32 位数据宽度
- 关联的可屏蔽中断支持
- 错误检测机制 (FIFO 满/空状态) FIFO 通道配置:
- 消息 FIFO 00: CPU1 → CPU0 (CPU1 只写, CPU0 只读) • 消息 FIFO 01: CPU1 → CPU0 (第二通道)
- 消息 FIFO 10: CPU0 → CPU1 (CPU0 只写, CPU1 只读)
- 消息 FIFO 11: CPU0 → CPU1 (第二通道)

这些 FIFO 配置允许双向通信，确保两个 CPU 可以异步发送和接收数据，而不会干扰彼此的执行流程。

此外，FIFO FULL (写阻塞) 和 FIFO EMPTY (读阻塞) 等错误标志为管理缓冲区状态提供了反馈。

Item		Description
Semaphore	Lock information	- Total of 16 factors - Security attributes and privileged attributes are selected every 8 factors
Inter-Processor interrupt	Non-maskable	- IPC0 1 factor, one non-maskable interrupt, security attributes and privileged attributes are selected. - IPC1 1 factor, one non-maskable interrupt, security attributes and privileged attributes are selected.
	Maskable	- IPC00 8 factors, one maskable interrupt, security attributes and privileged attributes are selected*¹ - IPC01 8 factors, one maskable interrupt, security attributes and privileged attributes are selected*¹ - IPC10 8 factors, one maskable interrupt, security attributes and privileged attributes are selected*¹ - IPC11 8 factors, one maskable interrupt, security attributes and privileged attributes are selected*¹
Message-FIFO	FIFO	- Message FIFO 00 (CPU1 -> CPU0), one maskable interrupt, security attributes and privileged attributes are selected*¹ - Message FIFO 01 (CPU1 -> CPU0), one maskable interrupt, security attributes and privileged attributes are selected*¹ - Message FIFO 10 (CPU0 -> CPU1), one maskable interrupt, security attributes and privileged attributes are selected*¹ - Message FIFO 11 (CPU0 -> CPU1), one maskable interrupt, security attributes and privileged attributes are selected*¹ 4 FIFO stages - Transfer data size 32 bits
	Error information	- Write with FIFO FULL*¹ - Read with FIFO Empty*¹

Figure 3. RA8 Dual-Core IPC Specification

3.1.4 RA8 双核 MCU IPC 寄存器架构

RA8 双核 MCU 提供全面的 IPC 机制，可在双核系统（CPU0: Cortex-M85 和 CPU1: Cortex-M33）之间实现安全、高效的通信和资源协调。寄存器级配置通过专用的 IPC 寄存器映射支持硬件信号量控制、中断信号传递和数据共享，如图所示。

硬件信号量控制（IPCSEM0 – IPCSEM15）

- 地址范围：0x4002_0000 至 0x4002_003C
- 提供 16 个独立的硬件信号量寄存器，允许每个核对共享硬件或内存的访问进行锁定。
- 这些寄存器通过使一个 CPU 获取锁来防止另一个 CPU 同时访问同一资源，从而实现互斥。

安全与特权控制

- IPCSAR（IPC 安全仲裁寄存器）– 0x4000_8610。管理访问控制和安全级别（安全区域与非安全区域）。
- IPCPAR（IPC 特权属性寄存器）– 0x4000_8614

o 定义资源访问的特权级别（特权与非特权）。

NMI 控制与状态寄存器

这些寄存器管理 CPU 之间的不可屏蔽中断（NMI）信号传递：

- IPC0NMICLR、IPC0NMISSET、IPC0NMISTA – CPU0 NMI 控制/置位/状态
- IPC1NMICLR、IPC1NMISSET、IPC1NMISTA – CPU1 NMI 控制/置位/状态
（地址范围：0x4002_0080 – 0x4002_009C）

它们用于触发或清除 NMI 请求，并检查中断状态以支持事件驱动通信。

核间通信通道

每个核都有自己的中断和基于 FIFO 的消息寄存器，用于数据传输和信号传递。

FIFO 寄存器（TXD、RXD）：

- 发送数据寄存器（只写）：
 - o CPU1 → CPU0: IPC0TXD0、IPC0TXD1
 - o CPU0 → CPU1: IPC1TXD0、IPC1TXD1
- 接收数据寄存器（只读）：o CPU0: IPC0RXD0、IPC0RXD1 o CPU1: IPC1RXD0、IPC1RXD1

FIFO 控制与状态寄存器：

- SET 寄存器：用于发送 IRQ 请求（如 IPC0ISET0、IPC1ISET1）
- CLR 寄存器：清除 IRQ 或状态（如 IPC0CLR0、IPC1CLR1）
- STA 寄存器：提供 FIFO 状态标志，如满/空或错误状态（如 IPC0STA0、IPC1STA1）

Address Map Summary:

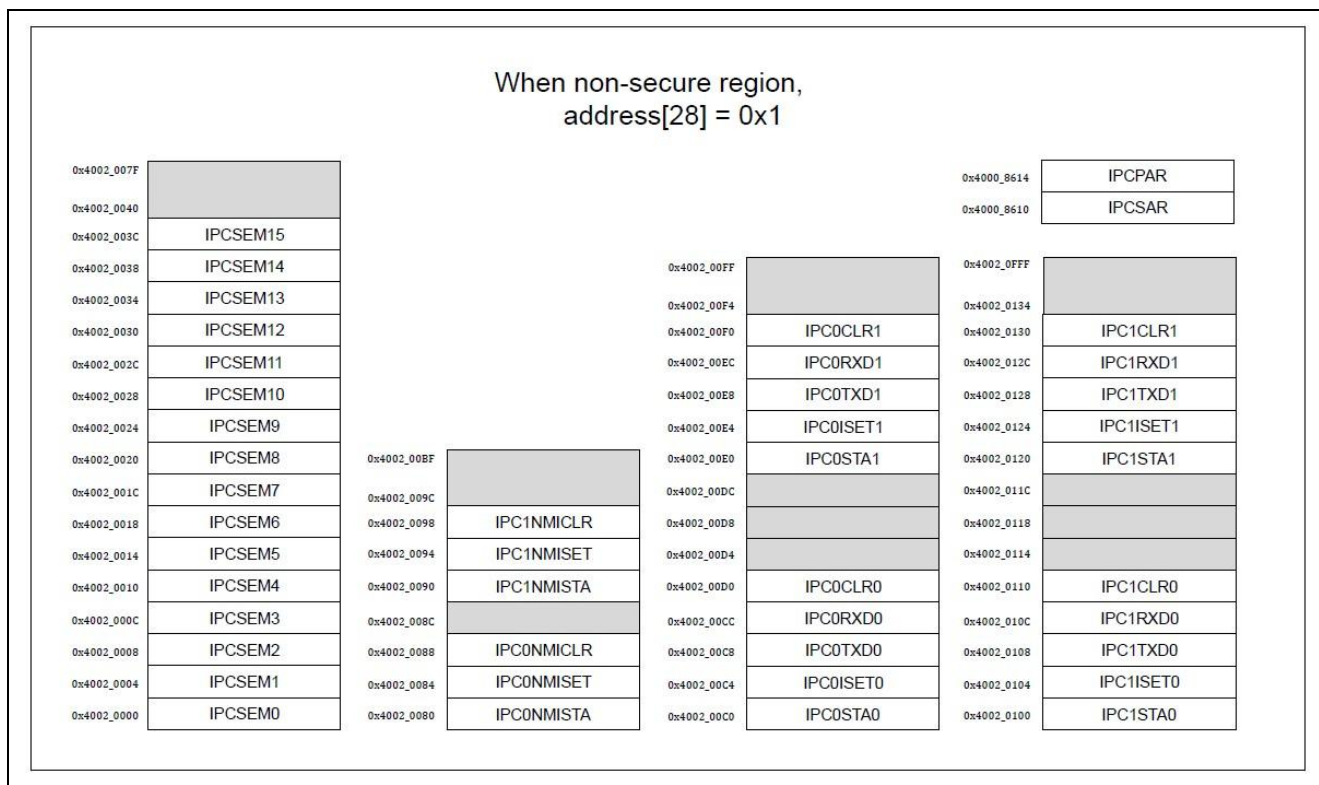


Figure 4. RA8 Dual-Core IPC Register Memory Map

3.2 使用 FSP 实现 IPC 模块

对于在 RA8 双核 MCU 上进行开发的应用开发者，硬件抽象层（HAL）提供了专用的 IPC 驱动和 API 套件。该驱动简化并标准化了在多核系统中实现多个 CPU 核之间通信的过程。

IPC HAL 提供了高级接口，抽象掉了直接与硬件寄存器、FIFO 和中断控制逻辑交互的复杂性。开发者可以借助简洁一致的 API 实现以下功能：

- 通过硬件支持的 FIFO 队列发送和接收消息，确保核间高效、低延迟的单向数据传输。
- 生成和处理可屏蔽中断，允许一个核异步通知另一个核有关事件或状态变化的信息。
- 设置用户定义的回调函数，以非阻塞、事件驱动的方式处理传入的消息或事件标志。

这种设计支持结构化的通信模式，促进了跨 CPU 核的更好模块化和同步，同时降低了竞态条件和时序问题的风险。通过隐藏硬件层面的细节，IPC HAL 实现了更快的开发周期和更便捷的维护，特别是在需要可靠核间协调的应用中，如工业自动化、实时控制和安全处理。

3.2.1 IPC API 概述：关键组件与数据结构

类别	项目	说明
API 函数	R_IPC_Open()	初始化并配置 IPC 实例。
	R_IPC_MessageSend()	通过硬件 FIFO 向另一个核发送消息。
	R_IPC_EventGenerate()	向目标核触发中断事件。
	R_IPC_CallbackSet()	为 IPC 事件注册用户定义的回调函数。
	R_IPC_Close()	关闭并释放 IPC 实例。
数据结构	ipc_callback_args_t	保存回调信息：通道、消息、事件和用户上下文。
	ipc_cfg_t	通道配置：ID、IRQ 号、优先级、回调函数。
	ipc_api_t	IPC 操作函数指针的结构体。
	ipc_instance_t	封装结构，保存控制块、配置和 API 指针。
	ipc_ctrl_t	内部用于跟踪 IPC 实例状态的不透明句柄。
被处理的事件	IPC_EVENT_IRQ0-7	来自另一个核的 IRQ 事件。
	IPC_EVENT_MESSAGE_RECEIVED	指示 FIFO 中接收到消息。
	IPC_EVENT_FIFO_ERROR_EMPTY	从空 FIFO 读取时发生错误。
	IPC_EVENT_FIFO_ERROR_FULL	向满 FIFO 写入时发生错误。
可触发的事件	IPC_GENERATE_EVENT_IRQ0-7	用于向另一个核触发 IRQ 事件。

类别	项目	说明
API 函数	R_BSP_IpcSemaphoreTake()	尝试获取（锁定）一个 IPC 硬件信号量
	R_BSP_IpcSemaphoreGive()	释放（解锁）指定的 IPC 硬件信号量。
	R_BSP_IpcNmiEnable()	使能当前核上的 NMI（不可屏蔽中断）。
	R_BSP_IpcNmiRequestSet()	向对端核发送 NMI 请求。

4. 同步与数据交换

RA8 双核 MCU 支持其双核之间健壮的同步与数据交换。它提供硬件 IPC 机制（如硬件信号量、中断和消息 FIFO）以确保安全高效的核间通信。硬件信号量用于对共享资源实施互斥访问。可屏蔽中断允许事件驱动的通信，而不可屏蔽中断（NMI）处理关键的跨核信号传递。消息 FIFO 使核间缓冲数据传输无需直接内存共享。这些机制通过 HAL API（如 R_IPC_MessageSend()、R_BSP_IpcSemaphoreTake() 和 R_BSP_IpcNmiRequestSet()）进行抽象。RA8 双核上的 FreeRTOS 支持允许开发者将 IPC 与 RTOS 原语集成，实现更有组织的协调。这些特性共同为双核运行提供了灵活、安全的框架。IPC 系统对于在 RA8 双核 MCU 上构建高性能、高响应性的多核应用至关重要。

4.1 双核通信的硬件信号量机制

RA8 双核 MCU 提供基于硬件的 IPC 硬件信号量（IPCSEmN），可在其双核架构（如 Cortex-M85 和 Cortex-M33）之间实现安全同步。硬件信号量通过允许一个核获取或释放信号量来确保互斥，从而防止竞态条件。R_BSP_IpcSemaphoreTake() 和 R_BSP_IpcSemaphoreGive() API 管理跨核的所有权。适用于保护共享资源，如内存缓冲区或外设。

4.2 IPC 的可屏蔽与不可屏蔽中断

RA8 双核支持可屏蔽中断（通过 IPC 事件如 IPC_EVENT_IRQ0）和 NMI 进行核间信号传递。可屏蔽中断允许使用灵活的事件位进行信号传递，并通过标准中断向量处理。NMI 用于紧急、高优先级的警报。API 如 R_BSP_IpcNmiEnable() 和 R_BSP_IpcNmiRequestSet() 便于核间 NMI 设置和请求触发。适用于常规协调和关键的跨核警报。

4.3 用于核间数据交换的消息 FIFO

RA8 双核上的 IPC 模块包含硬件 FIFO，可实现核间缓冲的异步消息传递。诸如 R_IPC_MessageSend() 等函数以及由 IPC_EVENT_MESSAGE_RECEIVED 触发的回调通过标准化的 HAL API 实现高效的单向通信，非常适合在不阻塞的情况下发送控制消息、命令或状态更新。

5. IPC 的 FSP 支持

FSP 为 IPC 提供专用的 HAL 级驱动模块，使开发者能够实现核间通信。这些驱动抽象了处理器核间硬件信号量、锁、消息传递和同步所需的底层寄存器交互。它们可以使用 e² studio 中的 FSP Configuration 工具轻松添加和配置。

Once added, these IPC drivers allow configuring parameters such as channel numbers, interrupt priorities, and callback functions. The following sections outline how to utilize these drivers effectively within application projects. This modular and configurable approach simplifies development while ensuring compatibility with the underlying hardware.

除了 HAL 级支持外，FSP 的 BSP 还包括硬件信号量组安全属性设置。这些设置定义了 IPC 相关硬件资源的安全或非安全访问权限，对于启用 TrustZone 的 MCU 尤为重要。在 HAL 和 BSP 两个级别上正确配置对于跨核安全可靠的 IPC 功能至关重要。

5.1 使用 FSP Configurator 设置 IPC

SummaryBSPClocksPinsInterruptsEvent LinksLinker SectionsStacksComponents

ProblemsConsoleProperties XSmart BrowserSmart ManualSearch

EK-RA8P1

Settings

Property	Value
> R7KA8P1KFLCAC	
▼ RA8P1	
series	8
▼ RA8P1 Device Options	
> OFS Registers	
▼ RA8P1 Family	
> SDRAM	
▼ Security	
> Exceptions	
> SRAM Accessibility	
> BUS Accessibility	
▼ IPC	
IPC Semaphore Group Security Attribution (0 to 7)	Secure State
IPC Semaphore Group Security Attribution (8 to 15)	Secure State
IPC NMI0 Security Attribution	Secure State
IPC NMI1 Security Attribution	Secure State
System Reset Request Accessibility	Secure State
System Reset Status Accessibility	Both Secure and Non-Secure State
Battery Backup Accessibility	Both Secure and Non-Secure State
Graphics Power Domain Security Attribution	Secure State

R7KA8P1KFLCAC

图 5. IPC NMI 与硬件信号量 - 安全属性设置概览

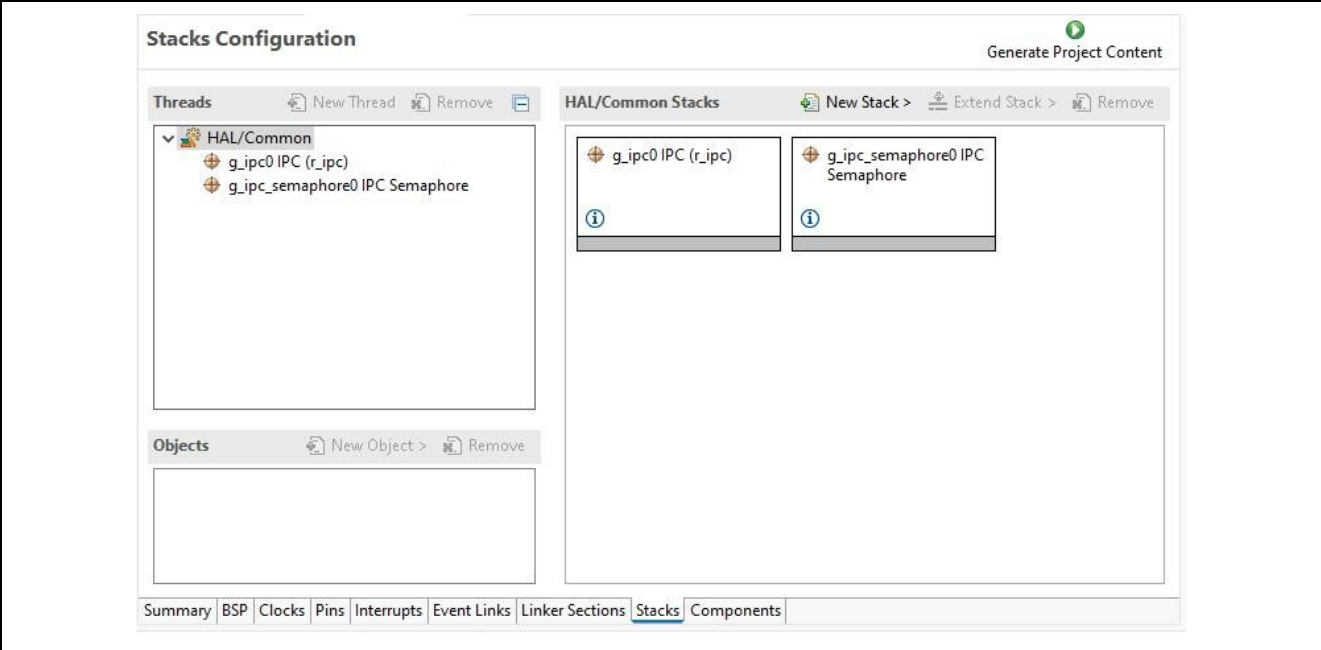


图 6. IPC 与硬件信号量模块配置

r_ipc 模块可通过 New Stack → System → IPC (r_ipc) 添加到 Stacks 选项卡中。通过右键单击 RA Configuration 工具中的模块并勾选"Non-secure Callable"复选框，可以为此模块生成非安全可调用保护函数。

Renesas FSP 中的 r_ipc 模块用于实现 RA8 双核 MCU 上 CPU0 与 CPU1 之间的核间通信。其主要功能包括发送消息和生成中断事件，以通知另一个核特定的条件或数据可用性。

- 该模块允许任一 CPU 核（CPU0 或 CPU1）使用 R_IPC_MessageSend() API 向对方发送消息，从而实现核间结构化通信和数据交换。
- 它还支持通过 R_IPC_EventGenerate() 函数生成事件。这会在接收核上触发可屏蔽中断，用于提醒另一个核有事件发生或有新数据可用。
- 为正确处理这些中断事件，开发者必须使用 Configurator 在驱动属性下配置回调函数。这会在接收核上注册一个回调函数，该函数在 IPC 事件触发时被调用。

使用注意事项：

- 如果使用中断/事件功能进行通信，配置回调函数至关重要，以确保接收核能够正确处理传入的事件。
- 这些 API 共同实现了核间高效、低延迟的信号传递和数据协调，这对于实时和多线程嵌入式应用至关重要。

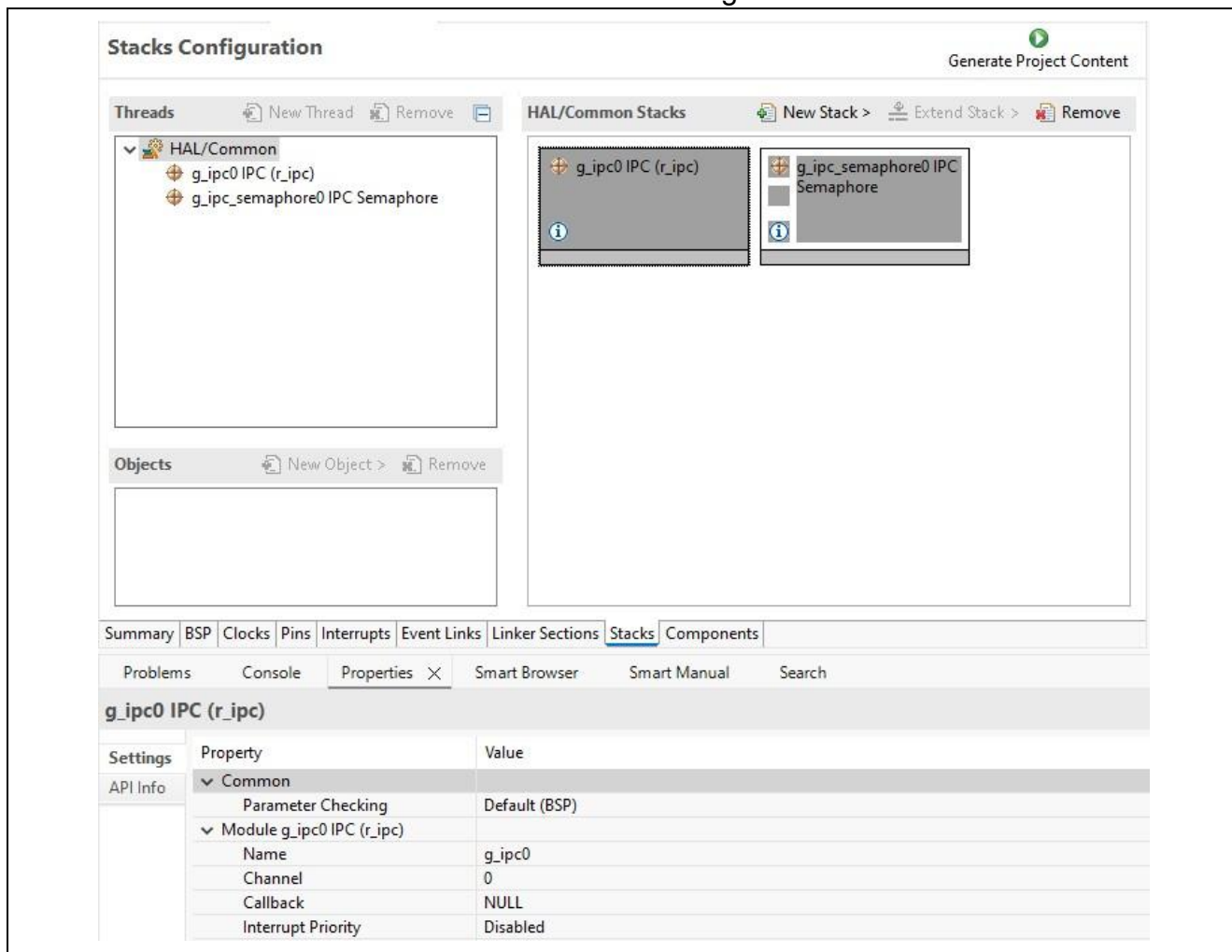


Figure 7. IPC Module Configuration

5.2 使用说明：IPC 中断与通道配置

在 Renesas FSP IPC 实现中，可屏蔽中断和 FIFO 共享同一条中断线。因此，IPC 驱动统一配置这两种机制，以确保跨通信类型的一致行为。

5.2.1 中断与回调行为

- 仅当 IPC 实例配置为接收消息或事件时，才需要中断和回调函数。
- 如果 IPC 仅用于发送，则无需设置中断或回调。
- 接收时，正确配置的回调函数至关重要。它将在响应中断时被调用，使接收核能够处理传入的消息或事件。

5.2.2 核间通道匹配

- IPC 通道必须在通信的两个核上一致配置。为确保可靠通信，一个核使用的通道号必须与另一个核上的配置完全匹配。
- 例如，要通过通道 0 实现核 0 到核 1 的通信：
 - 核 0 必须在通道 0 上配置并打开一个 IPC 实例。
 - 核 1 也必须在通道 0 上配置并打开一个 IPC 实例。
- 核间通道配置不一致将导致通信失败或未定义行为。

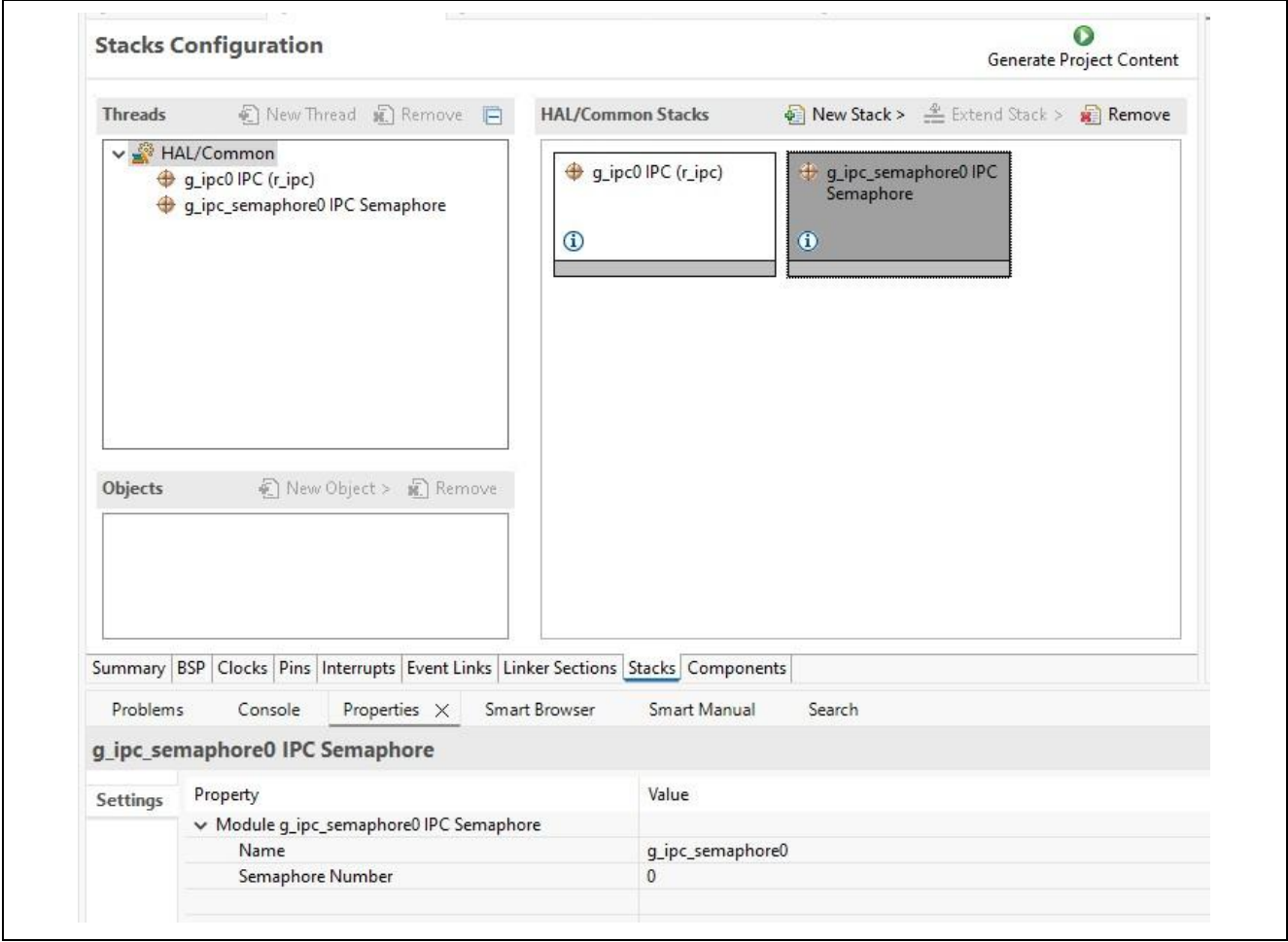


图 8. 硬件信号量模块配置

提供了一个专用的 IPC Semaphore Stack 模块，用于管理硬件信号量及其配置。

- 此模块可在 e² studio 中通过以下路径添加：
Stacks 选项卡 → New Stack → System → IPC Semaphore
- 配置模块时，用户必须选择特定的硬件信号量编号，该编号决定了使用哪个硬件信号量寄存器。

5.2.3 IPC 硬件信号量支持

FSP 支持的基于硬件的 IPC 硬件信号量可用于同步核间共享资源访问。

- 函数 R_BSP_IpcSemaphoreTake() 和 R_BSP_IpcSemaphoreGive() 分别用于获取和释放硬件信号量。
- 每个硬件信号量操作需要一个 semaphore_handle，该句柄必须关联一个有效的硬件信号量编号。

5.2.4 IPC 的 NMI 支持

Renesas FSP 支持核间 NMI，允许一个核向另一个核发送紧急、高优先级的中断请求。

- 为使能 IPC NMI 的接收，接收核必须调用 R_BSP_IpcNmiEnable()。此函数使能 NMI 并设置一个回调来处理该中断。
- 要从发送核触发对端核的 NMI，使用 R_BSP_IpcNmiRequestSet()。这将导致目标核立即发生不可屏蔽中断。
- 此机制适用于传递关键条件或紧急消息，这些消息不得因正常中断屏蔽而被延迟。

6. 示例工程与参考应用

本节提供示例代码、示例实现和参考工程，演示如何在 RA8 双核 MCU 上使用 IPC 功能。这些资源可作为开发者的实用指南，帮助他们理解 IPC 的设置和使用，并可直接使用或适配到他们自己的基于 RA8 双核的应用开发中。

6.1 IPC 实现示例代码

本示例代码演示如何使用 IPC 接口从核 0 向核 1 发送简单消息。

它演示了涉及的基本步骤：打开 IPC 实例、通过 FIFO 从核 0 发送消息，以及通过核 1 上的回调处理接收到的消息。这为在双核 RA MCU 中实现核间通信提供了清晰的起点。

核 0 示例

```
#define IPC_SEND_MESSAGE    0xABCD

void r_ipc_basic_messaging_core0_example (void)
{
    /* Open an IPC instance on the core 0 application configured to send a message to core 1.
     * @ref ipc_cfg_t::direction should be set to IPC_DIRECTION_SEND.
     * @ref ipc_cfg_t::channel should match for IPC instances on both cores.
     */
    fsp_err_t err = R_IPC_Open(&g_ipc0_ctrl, &g_ipc0_cfg);
    assert(FSP_SUCCESS == err);

    /* Send IPC_SEND_MESSAGE to core 1. */
    err = R_IPC_MessageSend(&g_ipc0_ctrl, IPC_SEND_MESSAGE);
    assert(FSP_SUCCESS == err);
}
```

核 1 示例

```

#define RESET_FLAG 0x00
#define SET_FLAG 0x01

uint32_t g_received_message = RESET_FLAG;
uint32_t g_is_message_received = RESET_FLAG;

/*****
 * @brief      IPC callback
 *
 * @param[in]  ipc_callback_args_t: The callback arguments.
 * @return     None
 *****/
void ipc_callback(ipc_callback_args_t *p_args)
{
    switch (p_args->event) {
        case IPC_EVENT_MESSAGE_RECEIVED:
        {
            g_received_message = (uint32_t)p_args->message;
            g_is_message_received = SET_FLAG;
            break;
        }
        default:
            break;
    }
}

void r_ipc_basic_messaging_core1_example (void)
{
    /* Open an IPC instance on the core 1 application configured to send a message to core 0.
     * @ref ipc_cfg_t::direction should be set to IPC_DIRECTION_RECEIVE .
     * @ref ipc_cfg_t::channel should match for IPC instances on both cores.
     */
    fsp_err_t err = R_IPC_Open(&g_ipc0_ctrl, &g_ipc0_cfg);
    assert(FSP_SUCCESS == err);

    /* Wait for event */
    while (!g_is_message_received)
    {
        ;
    }

    /* Message will be in g_message_received */
    (void) g_received_message;
}

```

以下代码演示了如何使用 IPC 接口通过可屏蔽中断从核 0 向核 1 触发事件。核 0 发送带有指定事件位的中断信号，而核 1 通过注册的回调监听并响应事件。此技术适用于核间无需发送数据的异步事件信号传递。

核 0 示例

```

void r_ipc_basic_interrupt_event_core0_example (void)
{
    /* Open an IPC instance on the core 0 application configured to send to core 1.
     * @ref ipc_cfg_t::direction should be set to IPC_DIRECTION_SEND.
     * @ref ipc_cfg_t::channel should match for IPC instances on both cores.
     */
    fsp_err_t err = R_IPC_Open(&g_ipc0_ctrl, &g_ipc0_cfg);
    assert(FSP_SUCCESS == err);

    /* Trigger interrupt on core 1 with event 0 bit set. */
    err = R_IPC_EventGenerate(&g_ipc0_ctrl, IPC_GENERATE_EVENT_IRQ0);
    assert(FSP_SUCCESS == err);
}

```

核 1 示例

```

#define false 0x00
#define true 0x01

uint32_t g_event0_triggered = false;

void r_ipc_basic_interrupt_event_core1_example (void)
{
    /* Open an IPC instance on the core 1 application configured to receive.
     * @ref ipc_cfg_t::direction should be set to IPC_DIRECTION_RECEIVE.
     * @ref ipc_cfg_t::channel should match for IPC instances on both cores.
     */
    fsp_err_t err = R_IPC_Open(&g_ipc0_ctrl, &g_ipc0_cfg);
    assert(FSP_SUCCESS == err);

    /* Wait for event */
    while (!g_event0_triggered)
    {
        ;
    }
}

void ipc_event_example_callback (ipc_callback_args_t * p_args)
{
    /* Check for message received event */
    if (IPC_EVENT_IRQ0 & p_args->event)
    {
        g_event0_triggered = true;
    }
}

```

6.2 RA8P1 MCU IPC 示例代码实现

本应用笔记随附的示例工程包演示了该器件上可用的多种 IPC 机制。这些工程旨在帮助开发者理解和尝试两个核之间交换数据和同步操作的不同方法：

1. 基于消息 FIFO 的 IPC
 - 其中一个工程展示消息 FIFO 作为 IPC 机制的使用。
 - 在此方法中，一个核将事件或 32 位数据放入 FIFO，而另一个核以同步方式取出。
 - 此方法特别适用于有序消息交换，可确保结构化的通信流程，无需复杂的握手逻辑。
2. 基于中断的 IPC
 - 其他示例工程演示了可屏蔽中断（IRQ）和不可屏蔽中断（NMI）作为 IPC 机制的使用。
 - 使用中断使一个核能够快速通知或向另一个核发送关于事件的信号，消除了轮询需求，确保低延迟响应。
 - 可屏蔽和不可屏蔽中断的使用为开发者提供了根据系统需求对 IPC 事件进行优先级排序的灵活性。
3. 基于硬件信号量的 IPC
 - 另一个示例工程重点展示 IPC 的硬件信号量功能。
 - 硬件信号量特别适用于协调两个核之间对共享资源的访问。
 - 通过使用硬件信号量，该工程演示了如何在两个核同时尝试访问或更新共享数据时实现互斥（mutex）并避免竞态条件。
4. 基于共享内存的 IPC
 - 最后，包含一个工程展示通过共享内存区域进行 IPC。
 - 在此机制中，两个核都读写公共内存区域，实现直接数据共享。
 - 该工程演示了如何将共享内存与硬件信号量或中断等同步机制结合使用，以确保跨核的数据完整性和一致性。

这些工程不仅演示了各种 IPC 机制，还为开发者提供了关于何时以及如何根据应用的实时要求、数据量和同步需求应用每种方法的实用参考。

6.3 在双核环境中使用 FreeRTOS 进行 IPC

FreeRTOS 提供标准的 RTOS 级 IPC 机制（如队列、信号量和消息缓冲区），用于单核内部通信。这些中间件对象常用于基于 FreeRTOS 的应用中，以高效管理任务间通信。

跨双核的 FreeRTOS IPC：概念与现实

虽然从技术上讲可以将 FreeRTOS IPC 机制扩展到双核系统，但由于 RA8 双核 MCU 的异构核架构，FSP 目前不支持此功能。在 RA8 双核 MCU 中，两个 CPU 核是不同的（例如 Cortex-M85 和 Cortex-M33），FreeRTOS 并非原生集成为支持跨两个核的共享 IPC 结构。

As a result, direct communication between FreeRTOS instances running on each core is not feasible using standard RTOS constructs. Instead, inter-core communication must rely on the MCU's hardware IPC mechanisms (e.g., hardware semaphores, events, and FIFO messaging via FSP IPC drivers).

替代方案：应用级 IPC 框架

为达到类似 FreeRTOS IPC 跨核使用的抽象级别和易用性，开发者可以创建自定义的应用级 IPC 框架。这些框架可以封装基于硬件的 IPC 功能，模拟 FreeRTOS 风格的消息传递和同步，在双核应用中实现更直观的核间通信。

这种方法有助于弥合高级 RTOS 设计模式与底层硬件 IPC 之间的差距，提高复杂嵌入式系统的可维护性和可扩展性。

为简化 RA8 双核 MCU 上的核间通信，可以开发基于 FreeRTOS 的封装框架。参考文档《使用 RA8 双核 MCU 开发 - R01AN7881EU》中的示例应用包含了一个示例框架。该框架旨在模拟标准 FreeRTOS IPC 对象（如队列和信号量），同时内部利用 MCU 的硬件 IPC 功能。

目的与优势

- 该封装允许开发者使用熟悉的类 FreeRTOS API 进行核间消息传递和同步。
- 它抽象了配置和管理硬件级 IPC（如硬件信号量、事件）的复杂性，呈现了更清晰、对应用更友好的接口。
- 此方法有助于将双核通信无缝集成到基于 FreeRTOS 的应用中，而无需修改 FreeRTOS 内部实现。

使用方法

- 开发者可以参考该封装框架，作为构建应用级 IPC 解决方案的参考或基础。
- 它可以作为中间件包含在工程中，与 FreeRTOS 任务一起使用以协调核间活动。

该框架弥合了 FreeRTOS 与硬件级 IPC 之间的差距，为双核嵌入式开发提供了高效、可扩展的解决方案。

7. 运行应用工程

7.1 导入工程

1. 启动 e² studio IDE。
2. 在 Workspace Launcher 中选择任意工作空间。
3. 关闭欢迎窗口。
4. 选择 File > Import。
5. 在 Import 对话框中选择 General > Existing Projects into Workspace。
6. 选择压缩文件"IPC_Via_Message_FIFO.zip"
7. 按下图所示选择 solution 工程和每个核上的开发工程示例，然后单击 Finish。

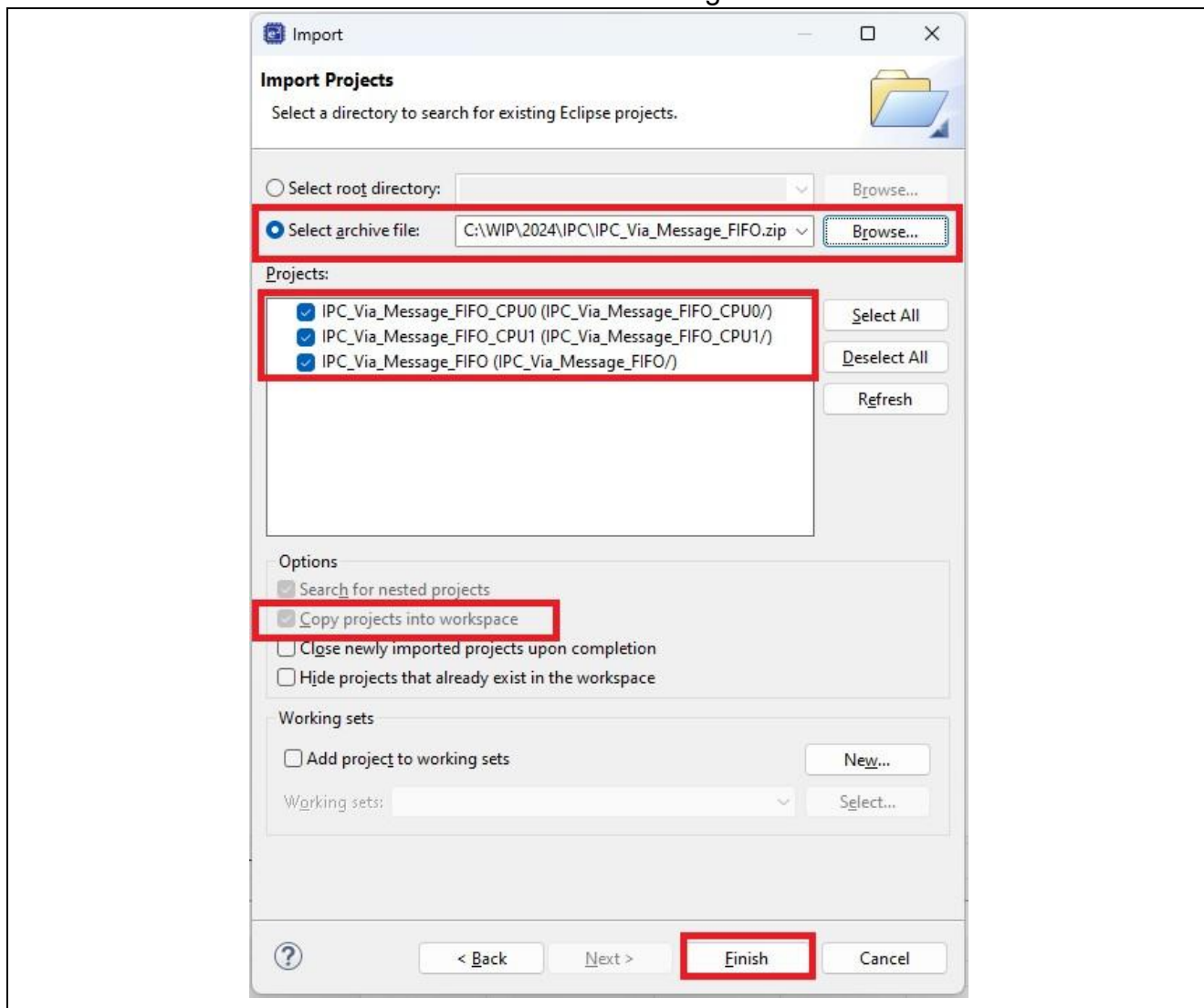


图 9. 导入基于消息 FIFO 的 IPC 工程

7.2 构建工程

从导入的工程包构建 solution 工程"IPC_Via_Message_FIFO"; 这将首先构建 CPU0 工程，然后构建 CPU1 工程。

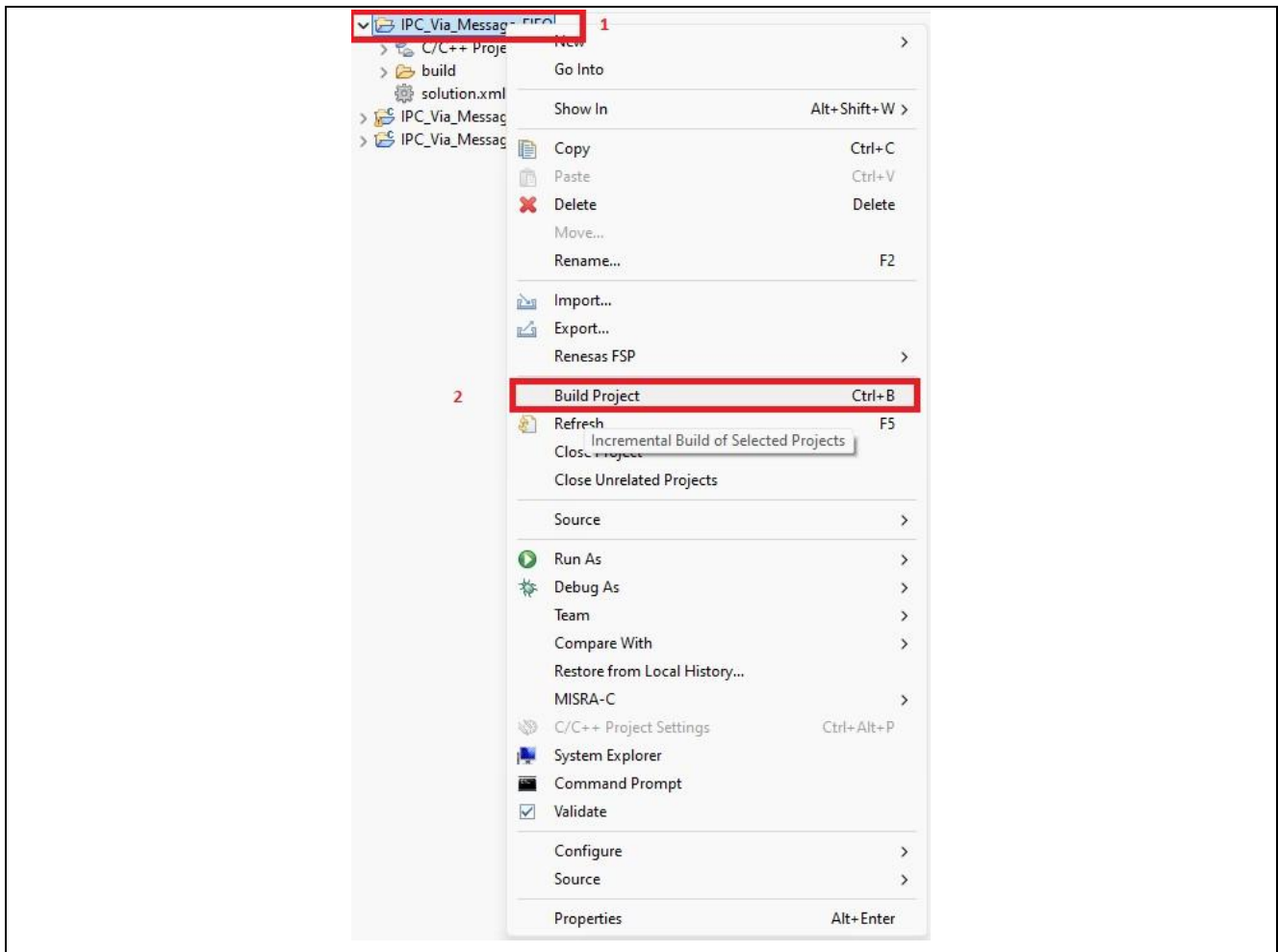


图 10. 构建基于消息 FIFO 的 IPC 工程示例

通过在 Build Log 控制台中验证构建状态，确保 CPU0 和 CPU1 工程均构建成功。

7.3 下载、运行与验证工程

初始时，器件必须初始化为 OEM_PL2 状态，无需 TrustZone 边界设置。

要同时启动两个核的调试：

1. 将 USB-C 线缆连接到调试接口（J10），另一端连接到主机 PC。这将建立 J-Link 连接用于下载镜像，并建立虚拟控制台（JLink CDC UART Port）用于通过串行终端进行用户交互。
2. 如图 11 所示打开 Debug Configurations。
3. 如图 12 所示选择"IPC_Via_Message_FIFO_CPU1 Debug_Multicore"。
4. 单击 Debug 启动双核调试会话。

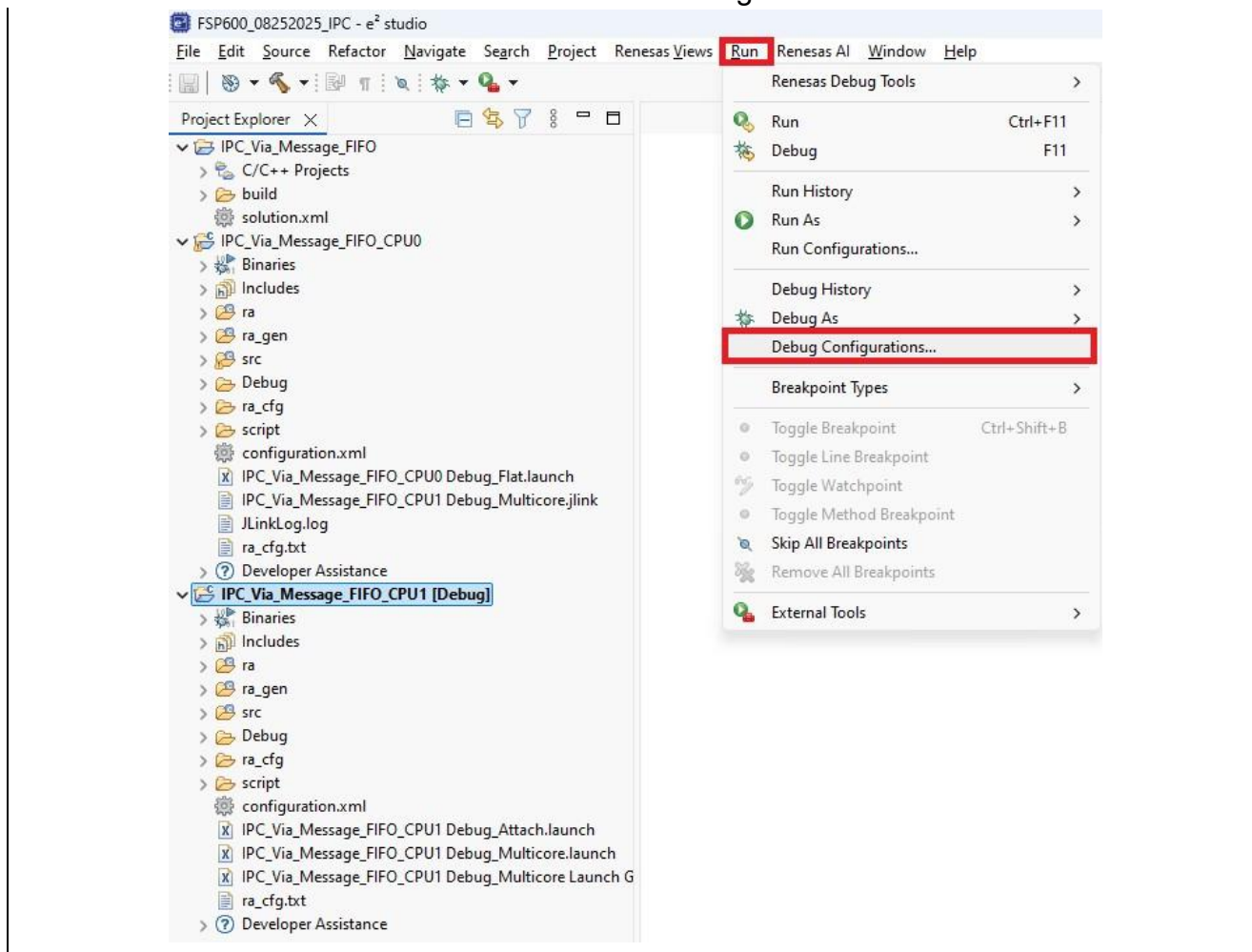


Figure 11. Example of Open Debug Configuration in IPC Via Message FIFO Example

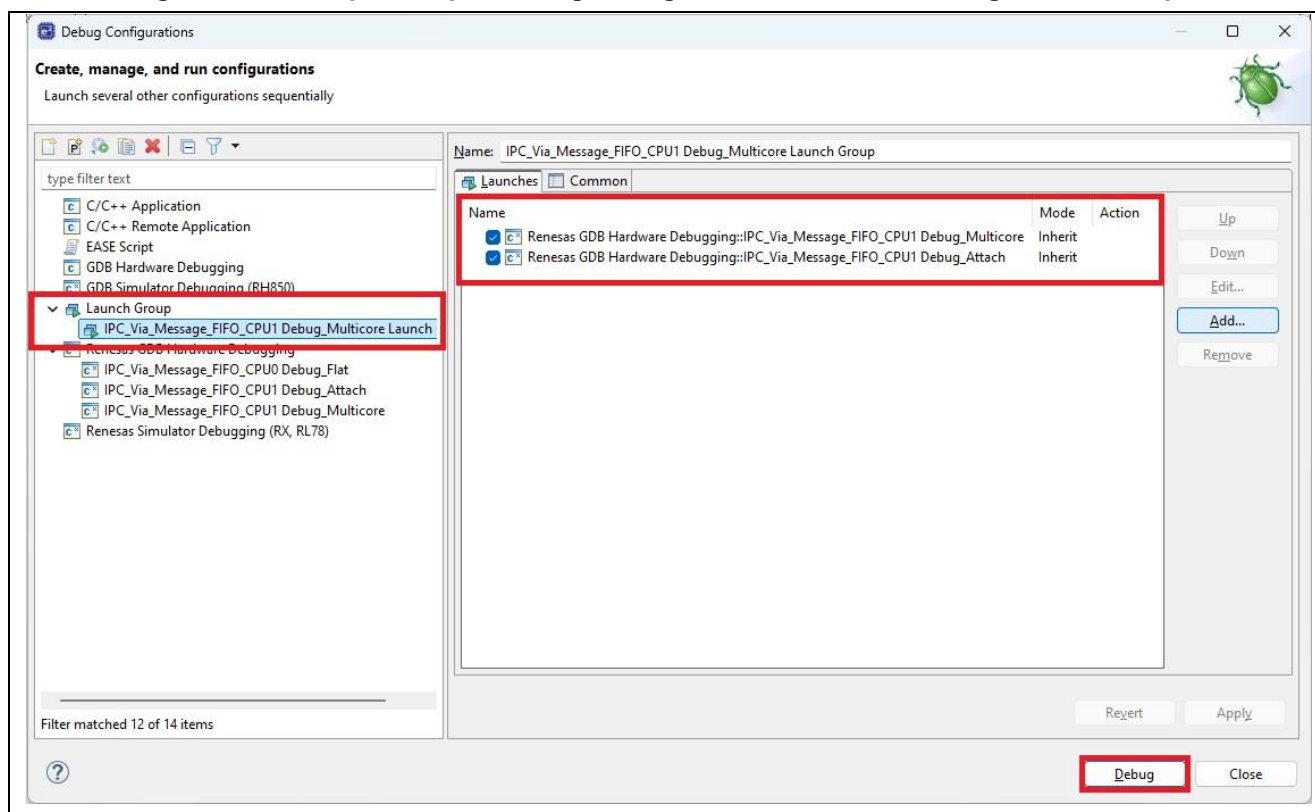


图 12. 调试基于消息 FIFO 的 IPC 示例

7.4 验证应用

打开串行终端（File → New connection → Serial），使用以下设置连接到 J-Link CDC UART Port：波特率 115200 bps、8 位数据位、无校验、1 位停止位、无流控。图 13 和图 14 说明了使用 Tera Term 终端的连接过程和配置步骤。

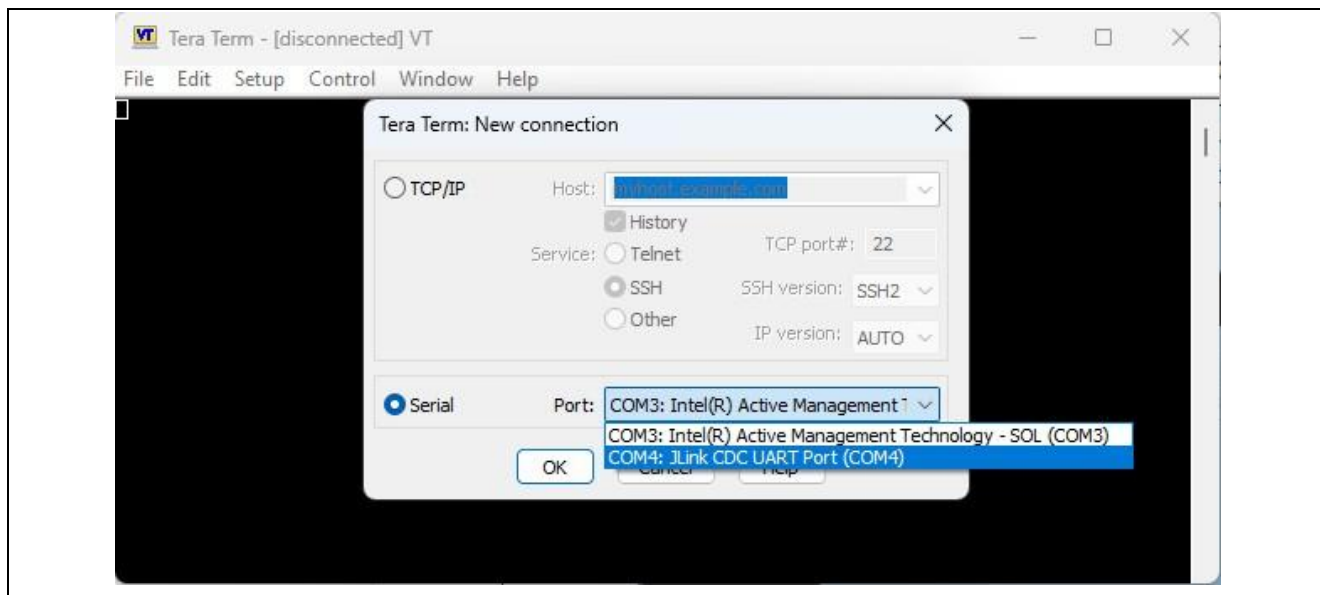


图 13. 连接到 JLink CDC UART Port 示例

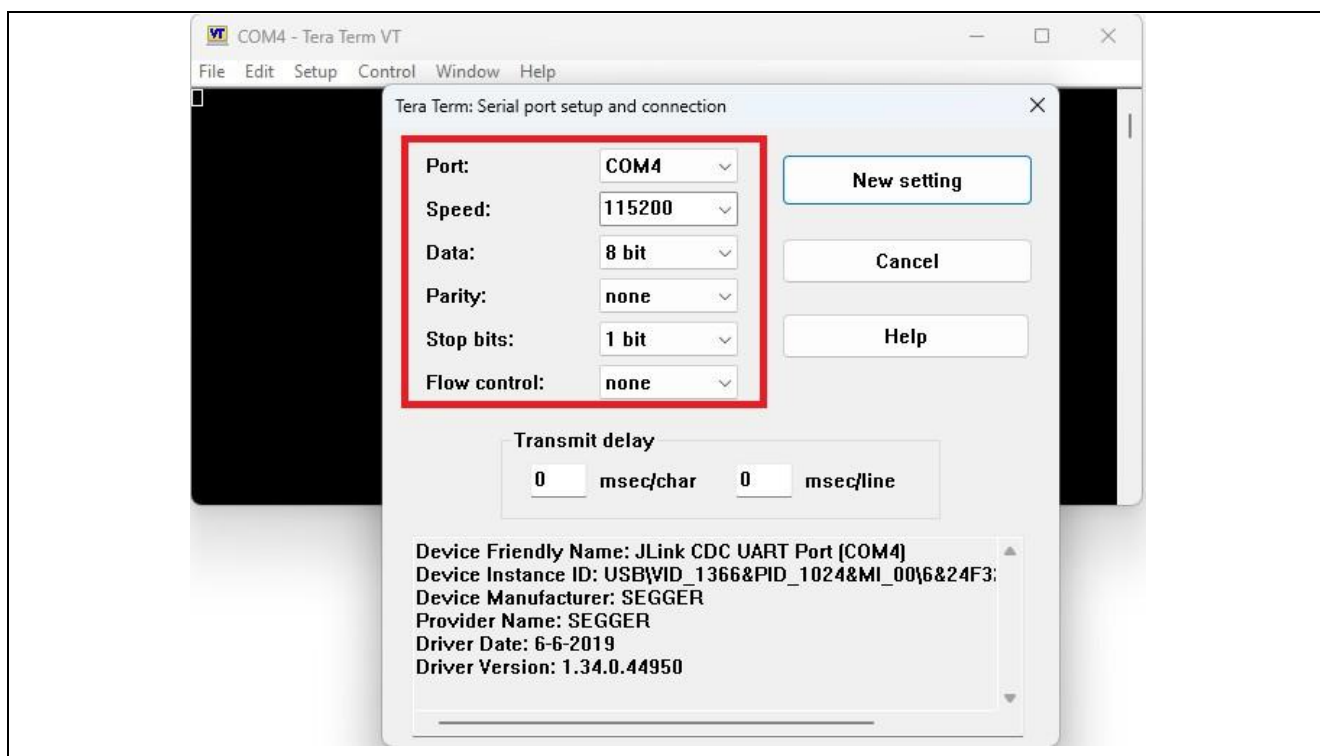
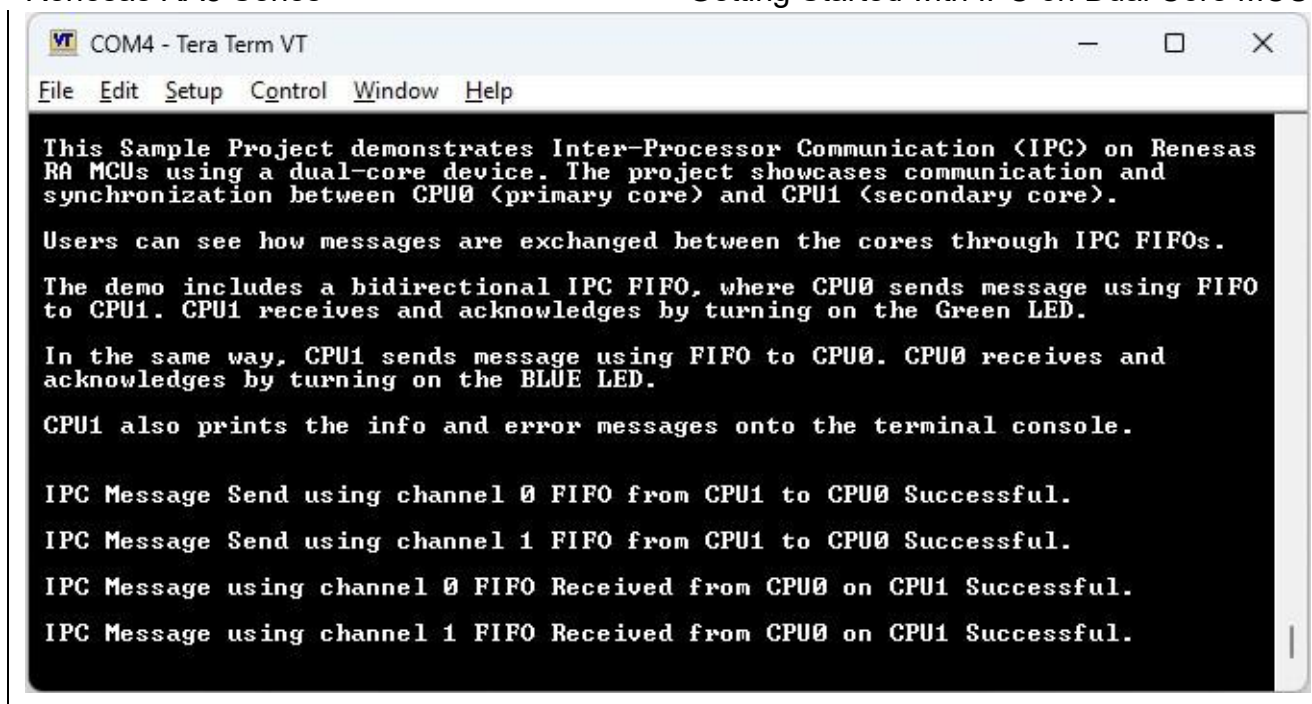


图 14. 设置 Tera Term 串行终端示例

返回 e² studio，按三次 Resume 以在两个核上执行应用。完成后，基于消息 FIFO 的 IPC 示例将向终端输出其状态，如图 15 所示。



```
COM4 - Tera Term VT
File Edit Setup Control Window Help

This Sample Project demonstrates Inter-Processor Communication (IPC) on Renesas
RA MCUs using a dual-core device. The project showcases communication and
synchronization between CPU0 (primary core) and CPU1 (secondary core).

Users can see how messages are exchanged between the cores through IPC FIFOs.

The demo includes a bidirectional IPC FIFO, where CPU0 sends message using FIFO
to CPU1. CPU1 receives and acknowledges by turning on the Green LED.

In the same way, CPU1 sends message using FIFO to CPU0. CPU0 receives and
acknowledges by turning on the BLUE LED.

CPU1 also prints the info and error messages onto the terminal console.

IPC Message Send using channel 0 FIFO from CPU1 to CPU0 Successful.
IPC Message Send using channel 1 FIFO from CPU1 to CPU0 Successful.
IPC Message using channel 0 FIFO Received from CPU0 on CPU1 Successful.
IPC Message using channel 1 FIFO Received from CPU0 on CPU1 Successful.
```

图 15. 基于消息 FIFO 的 IPC 示例成功演示的控制台日志

注意：工程包包含 4 个工程，用于演示各种 IPC 功能。用户可按照上述步骤导入、构建、运行和验证各工程。每个工程演示了 RA8P1 中可用的 IPC 的独特功能。

以下各节提供了示例工程的概览及其预期结果。每个工程突出展示了适用于双核 RA MCU 的一种独特 IPC 方法。虽然总体目标是演示两个核之间的通信，但每个工程侧重于不同的 IPC 机制，使用户能够了解可用选项的多样性。

1. IPC_Via_Event_Interrupt

此工程演示了使用标准中断事件作为 IPC 机制。

- CPU0 可以触发中断事件，由 CPU1 服务，反之亦然。
- 此方法适用于事件驱动同步，其中一个核在完成特定任务后向另一个核发出信号。
- 该工程展示了基于事件的信号传递如何以低开销实现及时通信。

2. IPC_Via_Message_FIFO

此工程利用消息 FIFO 进行核间数据传输。

- 一个 CPU 写入 32 位消息，另一个 CPU 读取，确保有序通信。
- FIFO 特别适用于需要排队并按顺序处理多条消息的场景。
- 此机制提供了在核间交换结构化数据流的可靠方式。

3. IPC_Via_NMI_Interrupt

此工程采用不可屏蔽中断作为 IPC 信号传递方法。

- 与事件中中断类似，NMI 允许一个 CPU 向另一个发出警报。
- 但与常规中断不同的是，NMI 不能被 CPU NVIC 禁用或屏蔽，确保关键的核间信号始终得到服务。
- 此机制对于紧急通信（如故障通知或安全关键信号传递）非常有价值。

4. IPC_Via_Shared_Memory

在此工程中，系统 RAM 的一个指定区域被保留为共享内存，两个核均可访问。

- CPU0 和 CPU1 可以直接读写此共享区域以交换数据。
- 共享内存提供最高的灵活性，支持批量数据传输、缓冲区共享和复杂数据结构。
- 可在其上叠加同步机制以确保数据一致性并避免冲突。

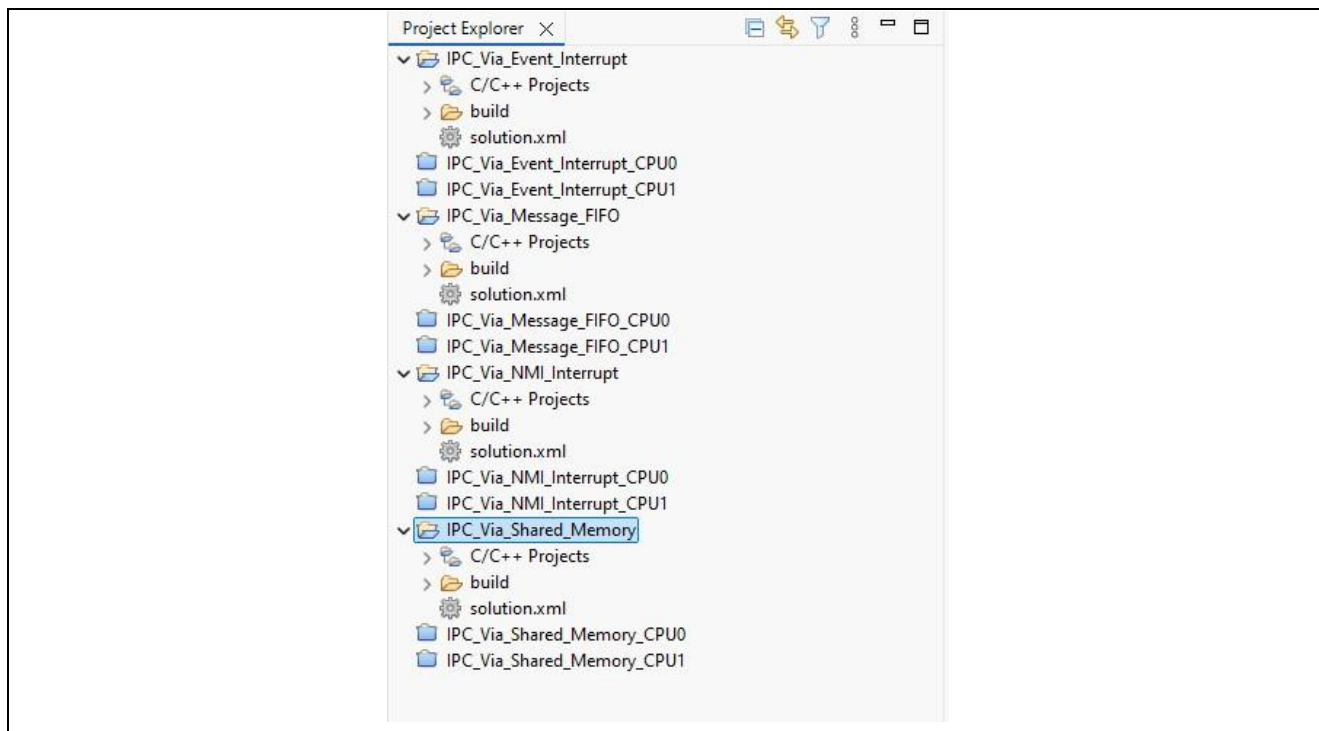


Figure 16. List of Projects in the bundle

用户可参考"IPC Via Message FIFO"示例运行并验证结果。以下截图显示了各示例工程的预期结果。

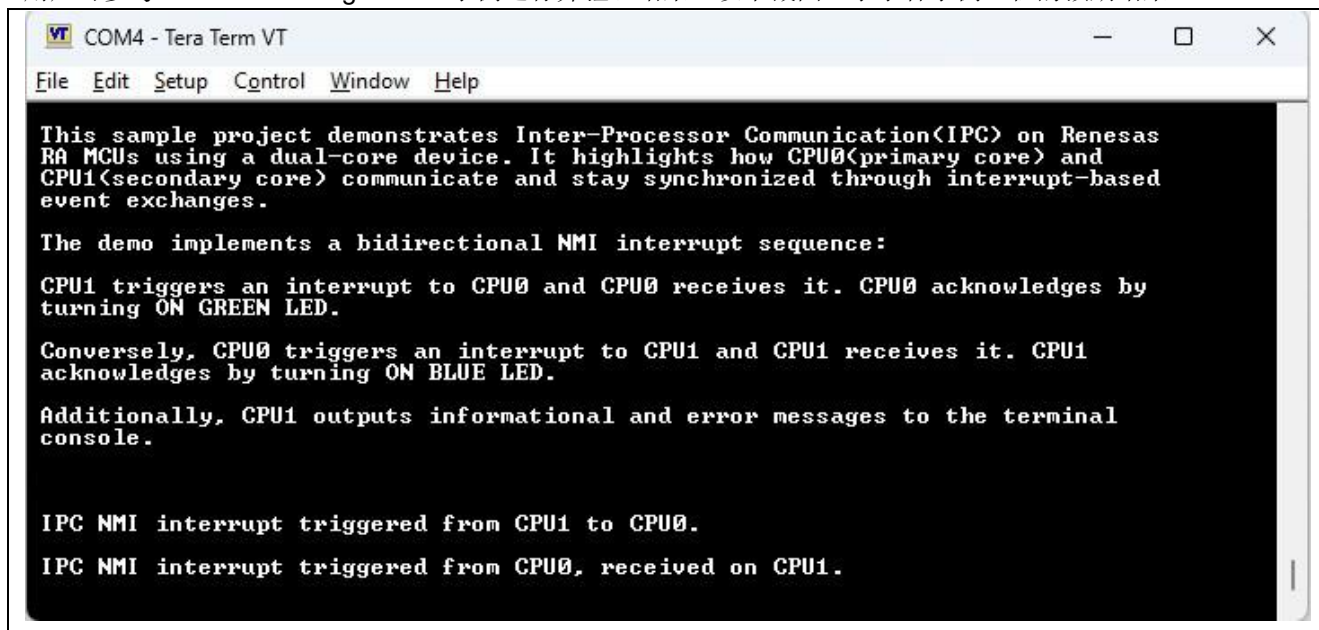


图 17. 基于 NMI 中断的 IPC 示例成功演示的控制台日志

```

COM4 - Tera Term VT
File Edit Setup Control Window Help

This sample project demonstrates Inter-Processor Communication(IPC) on Renesas
RA MCUs using a dual-core device. It highlights how CPU0(primary core) and CPU1
(secondary core) communicate and stay synchronized through shared memory data
exchanges.

The demo implements a bidirectional event interrupt sequence:

CPU1 triggers an interrupt to CPU0 and CPU0 receives it. CPU0 acknowledges by
turning ON BLUE LED.

Conversely, CPU0 triggers an interrupt to CPU1 and CPU1 receives it. CPU1
acknowledges by turning ON GREEN LED.

Additionally, CPU1 outputs informational and error messages to the terminal
console.

IPC00 <IPC0 channel 0> IRQ0 <interrupt event 0> triggered from CPU1 to CPU0.
IPC01 <IPC0 channel 1> IRQ1 <interrupt event 1> triggered from CPU1 to CPU0.
IPC10 <IPC1 channel 0> IRQ0 <interrupt event 0> triggered from CPU0, received on CPU1.
IPC11 <IPC1 channel 1> IRQ1 <interrupt event 1> triggered from CPU0, received on CPU1.

```

图 18. 基于事件中断的 IPC 示例成功演示的控制台日志

```

COM4 - Tera Term VT
File Edit Setup Control Window Help

This sample project demonstrates Inter-Processor Communication(IPC) on Renesas
RA MCUs using a dual-core device. It highlights how CPU0(primary core) and CPU1
(secondary core) communicate and stay synchronized through shared memory data
exchanges.

The demo implements a shared memory data exchanges from CPU0 <--> CPU1. CPU1
Writes a string data to dedicated shared memory allocated to CPU0 and CPU1.
CPU0 reads it from the shared memory and converts to upper case and writes it
back to shared memory. CPU1 reads it back and displays on to the Console

CPU1 triggers an interrupt to CPU0. CPU0 receives it and acknowledges by
turning ON BLUE LED.

Conversely, CPU0 triggers an interrupt to CPU1. CPU1 receives it and
acknowledges by turning ON GREEN LED.

Additionally, CPU1 outputs informational and error messages to the terminal
console.

*** String Data from CPU1 --> CPU0 ***
This is the Demo for showing IPC Shared Memory using RA8P1 !!!

IPC NMI interrupt triggered from CPU1 to CPU0.
IPC NMI interrupt received from CPU0 to CPU1.

*** String Data from CPU0 --> CPU1 ***
THIS IS THE DEMO FOR SHOWING IPC SHARED MEMORY USING RA8P1 !!!

```

图 19. 基于共享内存的 IPC 示例成功演示的控制台日志

8. 调试与故障排除

1. 要同时调试两个核，需要首先使用 Renesas Flash Programmer 或 Renesas Device Partition Manager 初始化 MCU。此步骤确保器件设置为保护级别 2，且 TrustZone 边界尚未配置。如果边界此前已设置，则必须将其重置以建立适合调试的环境。

完成此初始化至关重要；否则，在下载工程镜像或启动调试会话时可能会遇到问题。

2. 此外，使用 Launch group（组合各个启动配置）有助于运行和调试双核工程。

3. 此外，确保从 CPU0 工程调用 "R_BSP_SecondaryCoreStart()" 以启动 CPU1 核运行。出于调试目的，添加一个断点以确保此代码被调用，从而确认 CPU1 代码被调用。

9. 后续步骤

- 要了解有关 EK-RA8P1 套件的更多信息，请参阅 EK-RA8P1 网页 renesas.com/ek-ra8p1 上 Documents 和 Download 选项卡中分别提供的 EK-RA8P1 用户手册和设计包。同样，要了解 MCK-RA8T2、EK-RA8M2 和 EK-RA8D2，请参阅类似于 RA8P1 的各自网页。

- 要了解如何从头开始创建新的 e² studio 工程，请参阅 FSP 用户手册（renesas.com/ra/fsp）第 2 章 "Starting Development"。要了解如何使用 e² studio，请参阅 e² studio 网页（renesas.com/software-tool/e-studio）上提供的用户手册。

- Renesas 提供了多个示例工程，展示了 RA 和 MCU 的不同能力。

这些示例工程可作为用户开发自定义应用的良好起点。

- 请参阅应用笔记《使用 RA8 双核 MCU 开发》（文档号 R01AN7881EU），该文档演示了在单个工程中使用各种 IPC 功能。在此示例中，使用 RTC 和 ADC 外设演示 RA8P1 MCU 上的各种 IPC 能力。该应用工程是希望开发自己定制应用的用户的一个绝佳起点。

10. 参考文献

与双核内存架构和系统设计相关的基本参考、数据手册和文档集合。

可参考以下文档以更详细地了解 RA8 双核及其支持：

- Renesas FSP 用户手册：<https://renesas.github.io/fsp>
- Renesas RA 闪存编程：<https://www.renesas.com/us/en/document/apn/flash-memoryprogramming>
- Renesas RA MCU 数据手册：请访问 <http://renesas.com/ra> 并选择相关 MCU
- RA8P1、RA8M2、RA8T2 和 RA8D2 示例工程位于 Renesas RA GitHub：
<https://github.com/renesas/ra-fsp-examples>
- IPC 双核 MCU 入门指南
- Renesas RA 系列 RA8P1 用户手册：硬件篇（R01UH1064）
- Renesas RA 系列 RA8M2 用户手册：硬件篇（R01UH1066）
- Renesas RA 系列 RA8D2 用户手册：硬件篇（R01UH1065）
- Renesas RA 系列 RA8T2 用户手册：硬件篇（R01UH1067）

- Renesas RA 系列 RA8 快速设计指南 (R01AN7087) [R01AN7087](#))
- 利用带 Helium™ 的 CM85 核实现 RA8 MCU 高性能 (R01AN7127) [R01AN7127](#))
- EK-RA8P1、MCK-RA8T2、EK-RA8M2 和 EK-RA8D2 板卡设计原理图
- 使用 RA8 双核 MCU 开发 (R01AN7881) [R01AN7881](#))
- RA8P1 多核设置与双核 Hello World 运行 (R01AN7982)
- RA8P1 MCU 快速设计指南 (R01AN7883)
- RA8 双核 MCU 内存架构、配置与拓扑入门指南 (R01AN7880)
- Arm Cortex®-M85 处理器技术参考手册，文档号 101924，可从 Arm 获取

网站与支持

访问以下便捷 URL，了解 RA 系列的关键要素、下载组件及相关文档，并获取支持。

EK-RA8P1 资源	www.renesas.com/ek-ra8p1
MCK-RA8T2 资源	www.renesas.com/mck-ra8t2 www.renesas.com/mck-ra8t2
EK-RA8M2 资源	www.renesas.com/ek-ra8m2
EK-RA8D2 资源	www.renesas.com/ek-ra8d2
RA 产品信息	renesas.com/ra
RA 产品支持论坛	renesas.com/ra/forum
RA 灵活软件包	renesas.com/FSP
renesas.com/support	renesas.com/support

版本	日期	说明	
		页码	摘要
1.00	2025.07.30	-	初始版本
1.01	2025.10.16	-	新增对 RA8T2、RA8M2 和 RA8D2 的支持。

微处理单元与微控制器单元产品使用的一般注意事项

以下使用说明适用于所有 **Renesas** 微处理单元和微控制器单元产品。有关本文档所涵盖产品的详细使用说明，请参阅文档的相关章节以及可能已发布的任何产品技术更新。

1. 静电放电（ESD）防护措施

当 CMOS 器件暴露在强电场中时，可能导致栅氧化层损坏并最终降低器件性能。必须采取措施尽可能阻止静电的产生，并在静电产生时迅速将其消散。环境控制必须到位。在干燥环境中，应使用加湿器。建议避免使用容易积聚静电的绝缘体。半导体器件的存储和运输必须使用防静电容器、静电屏蔽袋或导电材料。所有测试和测量工具，包括工作台和地板，都必须接地。操作人员也必须使用腕带接地。不得裸手触摸半导体器件。对已安装半导体器件的印刷电路板也必须采取类似的防护措施。

2. 上电时的处理

产品在供电时的状态是不确定的。LSI 内部电路的状态不确定，寄存器设置和引脚的状态在供电时也是未定义的。在完成的产品中，当复位信号施加到外部复位引脚时，从供电到复位过程完成期间，引脚的状态不受保证。同样，对于通过片上上电复位功能复位的产品，从供电到电源达到规定复位电平期间，引脚的状态不受保证。

3. 断电状态下的信号输入

Do not input signals or an I/O pull-up power supply while the device is powered off. The current injection that results from input of such a signal or I/O pull-up power supply may cause malfunction and the abnormal current that passes in the device at this time may cause degradation of internal elements. Follow the guideline for input signal during power-off state as described in your product documentation.

4. 未使用引脚的处理

Handle unused pins in accordance with the directions given under handling of unused pins in the manual. The input pins of CMOS products are generally in the high-impedance state. In operation with an unused pin in the open-circuit state, extra electromagnetic noise is induced in the vicinity of the LSI, an associated shoot-through current flows internally, and malfunctions occur due to the false recognition of the pin state as an input signal become possible.

5. 时钟信号

复位后，只有在运行时钟信号稳定后才能释放复位线。在程序执行期间切换时钟信号时，需等待目标时钟信号稳定。当复位期间使用外部谐振器或外部振荡器生成时钟信号时，确保只有在时钟信号完全稳定后才能释放复位线。此外，当程序执行期间切换到由外部谐振器产生或由外部振荡器产生的时钟信号时，需等待目标时钟信号稳定。

6. 输入引脚的电压施加波形

Waveform distortion due to input noise or a reflected wave may cause malfunction. If the input of the CMOS device stays in the area between V_{IL} (Max.) and V_{IH} (Min.) due to noise, for example, the device may malfunction. Take care to prevent chattering noise from entering the device when the input level is fixed, and also in the transition period when the input level passes through the area between V_{IL} (Max.) and V_{IH} (Min.).

7. 禁止访问保留地址

禁止访问保留地址。保留地址是为将来可能的功能扩展而提供的。请勿访问这些地址，因为无法保证 LSI 的正确运行。

8. 产品之间的差异

在从一个产品更换为另一个产品（例如更换为不同型号的产品）之前，请确认此更换不会导致问题。同一系列但型号不同的微处理单元或微控制器单元产品的特性可能在内置存储器容量、布局模式和其他因素方面有所不同，这些差异可能影响电气特性范围，如特性值、工作裕度、抗噪声能力和辐射噪声量。更换为不同型号的产品时，应对该产品进行系统评估测试。

声明

1. 本文中提供的电路、软件和其他相关信息的描述仅用于说明半导体产品的操作和应用示例。您对电路、软件和信息在您的产品或系统设计中的纳入或任何其他使用承担全部责任。**Renesas Electronics** 对因使用这些电路、软件或信息而导致的您或第三方遭受的任何损失和损害不承担任何责任。
2. **Renesas Electronics** 特此明确声明，对因使用 **Renesas Electronics** 产品或本文中描述的技术信息（包括但不限于产品数据、图纸、图表、程序、算法和应用示例）而涉及或引起的第三方专利、版权或其他知识产权的侵权或任何其他索赔，不承担任何保证或责任。
3. 本文档不以明示、暗示或其他方式授予 **Renesas Electronics** 或其他方的任何专利、版权或其他知识产权的任何许可。
4. 您应负责确定需要从任何第三方获取哪些许可，并根据需要获取此类许可，以合法地进口、出口、制造、销售、使用、分销或以其他方式处置任何包含 **Renesas Electronics** 产品的产品。
5. 您不得对任何 **Renesas Electronics** 产品的全部或部分进行更改、修改、复制或反向工程。**Renesas Electronics** 对因此类更改、修改、复制或反向工程而导致的您或第三方遭受的任何损失或损害不承担任何责任。
6. **Renesas Electronics** 产品按以下两个质量等级分类："Standard"和"High Quality"。每个 **Renesas Electronics** 产品的预期应用取决于产品的质量等级，如下所示。

"Standard": 计算机；办公设备；通信设备；测试和测量设备；音视频设备；家用电器；机床；个人电子设备；工业机器人等。

"High Quality": 交通运输设备（汽车、火车、船舶等）；交通控制（交通信号灯）；大规模通信设备；关键金融终端系统；安全控制设备等。

除非在 **Renesas Electronics** 数据手册或其他 **Renesas Electronics** 文档中明确指定为高可靠性产品或适用于恶劣环境的产品外，**Renesas Electronics** 产品并非设计或授权用于可能对人类生命或人身造成直接威胁（人工生命支持设备或系统；外科植入物等）或可能导致严重财产损失（航天系统；海底中继器；核电控制系统；飞机控制系统；关键工厂系统；军事设备等）的产品或系统。**Renesas Electronics** 对因使用任何与 **Renesas Electronics** 数据手册、用户手册或其他 **Renesas Electronics** 文档不一致的 **Renesas Electronics** 产品而导致的您或任何第三方遭受的任何损失和损害不承担任何责任。

7. 没有任何半导体产品是绝对安全的。尽管 **Renesas Electronics** 硬件或软件产品中可能实施了安全措施或功能，**Renesas Electronics** 对任何漏洞或安全漏洞（包括但不限于对 **Renesas Electronics** 产品或使用 **Renesas Electronics** 产品的系统的任何未经授权的访问或使用）不承担任何责任。**RENESAS ELECTRONICS** 不保证或担保 **RENESAS ELECTRONICS** 产品或任何使用 **RENESAS ELECTRONICS** 产品创建的系统

将是无懈可击的或免受损坏、攻击、病毒、干扰、黑客攻击、数据丢失或盗窃或其他安全入侵（“漏洞问题”）。RENESAS ELECTRONICS 不承担任何和所有因任何漏洞问题引起的或与之相关的责任或义务。此外，在适用法律允许的范围内，

PERMITTED BY APPLICABLE LAW, RENESAS ELECTRONICS DISCLAIMS ANY AND ALL WARRANTIES, EXPRESS OR IMPLIED, WITH RESPECT TO THIS DOCUMENT AND ANY RELATED OR ACCOMPANYING SOFTWARE OR HARDWARE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE.

8. 使用 Renesas Electronics 产品时，请参考最新的产品信息（数据手册、用户手册、应用笔记、可靠性手册中的“半导体器件处理与使用的一般注意事项” Handling and Using Semiconductor Devices” in the reliability handbook, etc.），and ensure that usage conditions are within the ranges specified by Renesas Electronics with respect to maximum ratings, operating power supply voltage range, heat dissipation characteristics, installation, etc. Renesas Electronics disclaims any and all liability for any malfunctions, failure or accident arising out of the use of Renesas Electronics products outside of such specified ranges.
9. 尽管 Renesas Electronics 努力改善 Renesas Electronics 产品的质量和可靠性，但半导体产品具有特定的特性，如在一定使用条件下会以一定概率发生故障和失效。除非在 Renesas Electronics 数据手册或其他 Renesas Electronics 文档中明确指定为高可靠性产品或适用于恶劣环境的产品外，Renesas Electronics 产品不适用于抗辐射设计。您有责任实施安全措施，以防范在 Renesas Electronics 产品发生故障或失效时可能造成的人身伤害、火灾造成的伤害或损害和/或对公众的危险，例如硬件和软件的安全设计，包括但不限于冗余、火灾控制和故障预防、针对老化退化的适当处理或任何其他适当措施。由于单独评估微计算机软件非常困难且不切实际，您有责任评估您制造的最终产品或系统的安全性。
10. 有关各 Renesas Electronics 产品的环境兼容性等环保事项的详细信息，请联系 Renesas Electronics 销售办事处。您有责任仔细、充分地调查规范受控物质包含或使用的适用法律法规，包括但不限于欧盟 RoHS 指令，并在使用 Renesas Electronics 产品时遵守所有此类适用法律法规。Renesas Electronics 对因您不遵守适用法律法规而造成的任何损害或损失不承担任何责任。
11. Renesas Electronics 产品和技术不得用于或被纳入其制造、使用或销售被任何适用的国内或外国法律法规禁止的任何产品或系统。您应遵守对当事方或交易拥有管辖权的任何国家政府颁布和执行的任何适用的出口管制法律法规。
12. Renesas Electronics 产品的买方或经销商，或向第三方分销、处置或以其他方式销售或转让该产品的任何其他方，有责任事先告知该第三方本文件中规定的内容和条件。
13. 未经 Renesas Electronics 事先书面同意，不得以任何形式全部或部分重印、复制或翻印本文件。 14. 如果您对本文件中包含的信息或 Renesas Electronics 产品有任何疑问，请联系 Renesas Electronics 销售办事处。

（注 1） 本文件中使用的“Renesas Electronics”是指 Renesas Electronics Corporation，也包括其直接或间接控制的子公司。（注 2）
“Renesas Electronics product(s)”是指由 Renesas Electronics 或其开发或制造的任何产品。

（Rev.5.0-1 2020 年 10 月）

Corporate Headquarters

TOYOSU FORESIA, 3-2-24 Toyosu, Koto-ku,
Tokyo 135-0061, Japan www.renesas.com

Trademarks

Renesas and the Renesas logo are trademarks of Renesas Electronics Corporation. All trademarks and registered trademarks are the property of their respective owners.

Contact information

For further information on a product, technology, the most up-to-date version of a document, or your nearest sales office, please visit:
www.renesas.com/contact/.

© 2025 Renesas Electronics Corporation. 保留所有权利